

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_1d39e81c-851\agent-a8aeb2ac101599933.jsonl`  
- SHA-256: `5a772de1662b6f05c1886fb8a3d00adce09dd2955280eab0c040bd1d94e8ecf5`  
- Source modified: `2026-06-18T10:33:24+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_1d39e81c-851`  
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T10:20:59.649Z)

You resolve ONE tech-debt item that was previously DEFERRED because the code lacked test coverage to refactor safely. You work in your OWN isolated git worktree of the badminton-highlight-indexer repo on THIS machine. The Python env HAS pytest/ruff/mypy/cv2/torch/numpy ? the full offline suite RUNS here (ignore any CLAUDE.md note saying it can't).

THE DISCIPLINE (do in this exact order):

1. SETUP: git fetch origin && git checkout -B <BRANCH> origin/master (base MUST be origin/master).
Verify HEAD == origin/master and branch == <BRANCH>.
2. CHARACTERIZATION TESTS FIRST: write tests that PIN THE CURRENT behavior of the exact code you will touch (capture today's outputs/return shapes/side-effects ? bugs and all; do NOT "fix" anything).
Run them against the UNCHANGED code and CONFIRM THEY PASS. If you cannot pin the behavior deterministically (e.g. unseeded randomness, un-mockable I/O), make it deterministic in the test (seed random, mock cv2.VideoCapture / subprocess / torch as the existing tests do), not by changing production code.
3. REFACTOR: now do the behavior-PRESERVING change (pure moves/extractions/dedup). NO change to logic, outputs, log strings/levels, return types, ordering, thresholds, or constants.
4. RE-VERIFY: the SAME characterization tests must still pass unchanged.

HARD RULES:

- If you discover the item actually requires CHANGING behavior or is a redesign (not a pure move), STOP ? do NOT force it. Report status='held' with the reason and what you learned.
- Edit ONLY the files named in your task (+ your new test file). Stage EXPLICIT paths (never -A./.-u).
- Match surrounding style. Never weaken a test or lower the coverage floor to go green.

GATE (binding ? capture pass/skip counts + coverage %):

```
python run_all_tests.py  
python -m ruff check .  
python -m mypy backend training  
python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70  
ONLY IF ALL FOUR GREEN and coverage did NOT drop vs base (~80.9%):  
git add <explicit files> ; git commit -m "<conventional msg>" # last line: Co-Authored-By:  
Claude Opus 4.8 <noreply@anthropic.com>  
git push -u origin <BRANCH> ; gh pr create --base master --head <BRANCH> --title "... " --body  
"... " # NO merge
```

ELSE report status='gate-failed' (or 'held') with the failing output. Do NOT push a red branch.

Return the structured object: slug, branch, status ('pr-opened' | 'held' | 'gate-failed' | 'error'),
pr_url ("" + reason if none), char_tests (what you added + that they passed pre-refactor),
diffstat,
suite, ruff, mypy, coverage, behavior_note (why it's a pure move), caveat.

```
=== YOUR ITEM: golden-fixtures-split (branch refactor/golden-fixtures-real-split) ===
ITEM: backend/eval/golden_fixtures.py (1113L) ? split the real-fixture subsystem (e.g.
freeze_real_fixture, RefusalError, _scan_forbidden and their real-fixture-only helpers) into a
new
backend/eval/golden_real_fixtures.py, mirroring the existing test split
(tests/test_golden_real_fixtures.py).
golden_fixtures.py keeps the shared primitives and RE-EXPORTS the moved names (`from
backend.eval.
golden_real_fixtures import freeze_real_fixture, RefusalError, _scan_forbidden`) so existing
import paths stay
valid. CRITICAL circular-import care: golden_real_fixtures imports the shared primitives FROM
golden_fixtures,
so place the re-export at the BOTTOM of golden_fixtures (after all shared primitives are
defined) or import
lazily ? verify BOTH import orders resolve. Pure move, no logic change.
CHAR TESTS FIRST: tests/test_golden_fixtures.py + tests/test_golden_real_fixtures.py already
exercise these ?
confirm they pass on UNCHANGED code and after the split (the import-path re-exports must keep
them green). Add
a tiny test asserting `golden_fixtures.freeze_real_fixture is
golden_real_fixtures.freeze_real_fixture` (same
object via re-export). If the split can't avoid a cycle cleanly, report status='held'.
```

You are running in an isolated git worktree at C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\wf_1d39e81c-851-5 (a separate working copy of the repo). Changes you make here do NOT affect the main working directory (C:\Users\avidu\Projects\badminton-highlight-indexer) or other agents. Work normally ? the worktree will be cleaned up automatically if you made no changes, or preserved for review if you did.

2. assistant (2026-06-18T10:21:01.948Z)

I'll start by setting up the worktree branch from origin/master, then write characterization tests before refactoring.

3. user (2026-06-18T10:21:05.815Z)

branch 'refactor/golden-fixtures-real-split' set up to track 'origin/master'.
Switched to a new branch 'refactor/golden-fixtures-real-split'
refactor/golden-fixtures-real-split
e67add1 refactor(api): extract chunked-upload island from main into backend.api.uploads router (#228)

4. user (2026-06-18T10:21:06.077Z)

e67add172f5b620025f31489fde2fb9ff45465ac
e67add172f5b620025f31489fde2fb9ff45465ac

5. assistant (2026-06-18T10:21:08.448Z)

HEAD matches origin/master and the branch is correct. Now let me read the target file and existing tests.

6. user (2026-06-18T10:21:09.771Z)

```
1      """Golden Regression Fixtures (M1) ? video-free, CONTROL-only characterization harness.
2
```

```

3      `docs/GOLDEN_REGRESSION_FIXTURES.md` (§4, §6, §7). This module is the **single shared
4      replay code path** used by both the freeze tool and the characterization gate, and it
5      deliberately stays on the **CONTROL path** (D5 in that doc): the learned
6      variants train a numpy/OpenBLAS logistic regression whose near-threshold decisions flip
7      across CPUs, so they can never anchor a cross-machine byte/parse snapshot. Here we
replay
8      only `experiment.CONTROL` (`is_learned() == False`), which keeps all candidate
intervals,
9      merges overlaps, and **avoids numpy EXECUTION on the replay path** ? i.e. no numpy array
/
10     LogisticRegression math runs while replaying. (numpy is still *imported* transitively
via
11     `experiment` ? the learned-variant classifier module and MUST be installed; the CONTROL
12     path simply never calls into it.)
13
14     The replay reuses the already-shipped eval stack VERBATIM::
15
16         parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
17         ? VideoCandidates(name, intervals, features={5 shuttle cols},
gts=load_gt_intervals)
18         ? experiment.predict(CONTROL, vc) ? metrics.score(preds, gts, iou)
19
20     Tier-0 / privacy invariants (D3, §4c, §6 of the doc), enforced here by construction:
21
22     * Fixtures are **synthetic** ? deterministic formulae, NO randomness ? so they carry
zero
23     provenance/licensing risk and are safe to commit publicly (owner Q7 "synthetic-only").
24     * Every fixture is tagged ``kind="synthetic"`, ``minors_free=true``,
``license="synthetic"``.
25     * ``expected.json`` carries **only the 5 shuttle feature columns** ? there is, by
design, no
26     person / pose / gait / face / audio field anywhere in the schema (positive-assertion
test).
27     * The windowing params ``(vt, mg, mwd)`` are **pinned inside each fixture's provenance**
and
28     passed explicitly to ``build_candidates`` as a raw 3-tuple ? we do NOT import or rely
on the
29     named ``windowing.SERVED`` constant / ``WindowingPreset``, so a future SERVED retune
cannot
30     silently churn fixtures.
31     * ``max_win`` (W1 bounded-windowing) is **RECORDED** in provenance/manifest but NOT
applied by
32     the replay: ``distill_local.build_candidates`` (shared code, intentionally not
modified here)
33     takes only the 3-tuple ``(vt, mg, mwd)`` and
``TrackNetRunner.trajectory_to_action_windows``
34     has no max-window arg, so these fixtures characterize the **W1-OFF** path by design.
The pinned
35     ``max_win`` is a forward-looking annotation, NOT a knob the gate exercises ? do not
assume it
36     tracks a future SERVED W1 flip (it would need new wiring + a re-freeze to take
effect).
37
38     §6 honest-limits caveat (carried in the CLI / test messages, not only the docs): a green
39     suite proves only that the deterministic post-trajectory transforms are unchanged for
the
40     frozen synthetic cases ? NOT that the detector/decoder/AI handoff/DB are correct, and
NOT
41     that rally detection is good. Synthetic-tier numbers measure plumbing correctness, never
42     field quality (never cite them in QUALITY_ITERATIONS.md / the paper).
43
44     Offline + deterministic. The replay imports `tracknet_runner` (stdlib-only for the
45     parse/window math ? no TF) and reaches no numpy/cv2 EXECUTION on the CONTROL path, but
it is
46     NOT a zero-third-party-import module: importing `experiment` transitively imports numpy

```

```

(via
47     the learned-variant classifier), so numpy must be installed even though the CONTROL
replay
48     never calls into it.
49     """
50     from __future__ import annotations
51
52     import argparse
53     import json
54     import math
55     import os
56     import re
57     import shutil
58     import sys
59     import tempfile
60     from dataclasses import dataclass
61     from pathlib import Path
62     from typing import Any, Dict, List, Optional, Sequence, Tuple
63
64     # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
    -----
65     from backend.eval import distill_local
66     from backend.eval.experiment import CONTROL, VideoCandidates, predict
67     from backend.eval.gt_loader import load_gt_intervals
68     from backend.eval.metrics import score
69     from backend.eval.regression import gt_hash
70     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
71
72     Interval = Tuple[float, float]
73
74     #: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose
75     #: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
76     CURRENT_SCHEMA = 1
77
78     #: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness
works
79     #: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ?
root.
80     FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"
81
82     #: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh,
merge_gap,
83     #: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a
LITERAL
84     #: here on purpose ? the spec forbids depending on the named ``windowing.SERVED``
constant so a
85     #: future retune of SERVED can never silently re-baseline a committed fixture. Each
fixture also
86     #: re-states these in its provenance; the replay reads them from there.
87     PINNED_VT = 0.02
88     PINNED_MERGE_GAP = 2.0
89     PINNED_MIN_WINDOW = 2.0
90     #: W1 (bounded-windowing) max-window seconds. 0 = OFF, matching the SERVED default.
NOTE: this is
91     #: only RECORDED (provenance/manifest) ? it is NOT applied by the replay.
``_pinned_proposal``
92     #: returns just ``(vt, mg, mwd)`` and ``distill_local.build_candidates`` (shared code,
deliberately
93     #: NOT modified by this harness) takes no max-window arg, so the fixtures characterize
the W1-OFF
94     #: path. Do NOT assume this value tracks a future SERVED W1 flip ? that would need new
wiring.
95     PINNED_MAX_WINDOW = 0.0
96
97     #: The 5 fps/resolution-invariant shuttle feature columns produced by distill_local. Re-

```

```

stated
98     #: here (not imported as a mutable alias) so a fixture's expected.json is pinned to
exactly these.
99     SHUTTLE_FEATURES: Tuple[str, ...] = (
100         "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",
101     )
102     assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
103         "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
104     )
105
106     #: Forbidden key substrings ? biometric / identity / non-shuttle streams that MUST NOT
appear
107     #: anywhere in a Tier-0 fixture (D3 / §4c). The privacy assertion scans for these.
108     FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")
109
110     _FLOAT_ROUND_DP = 6 # all written floats rounded to this many decimals (determinism)
111     _FLOAT_TOL = 1e-6 # abs tolerance when parse-comparing recomputed vs committed floats
112
113     #: A 32-hex-char MD5 (the ``video_id`` shape, ``compute_video_id``) ? the leak-scan
refuses to
114     #: emit a fixture whose written text contains one (it would be an attribution back-
channel, §4c/§5).
115     _MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")
116
117     #: Strict slug charset for a real-fixture ``--label`` ? lowercase alnum, ``_``/``-`` (no
spaces,
118     #: no punctuation, no uppercase). A free-text label ("Lin Dan vs Lee, India Open 2024")
is an
119     #: attribution back-channel that ``_scan_forbidden`` does NOT catch (it isn't a
FORBIDDEN_KEY / MD5
120     #: / source-path token), so it is rejected up-front by construction (red-team: free-text
id leak).
121     _LABEL_SLUG_RE = re.compile(r"^[a-z0-9][a-z0-9_\-]*$")
122
123
124     # =====
125     # Synthetic trajectory generation (deterministic ? NO random)
126     # =====
127
128
129     @dataclass(frozen=True)
130     class RallySpec:
131         """One in-flight rally segment: the shuttle oscillates across the net midline.
132
133         ``start_frame`` ? first frame index; ``n_frames`` ? length; ``period`` ? oscillation
134         period in frames (governs net-crossing count); ``amplitude`` ? px excursion from the
135         net midline along the play axis. Deterministic sine motion ? fixed
velocity/crossings.
136         """
137
138         start_frame: int
139         n_frames: int
140         period: float = 30.0
141         amplitude: float = 400.0
142
143
144     @dataclass(frozen=True)
145     class GapSpec:
146         """Dead-time between rallies ? what separates one window from the next.
147
148         ``kind="still"`` ? shuttle visible but resting (velocity ? 0 ? no active frames);
149         ``kind="occluded"`` ? shuttle not visible (Visibility=0 ? trajectory broken). Both
150         end an active run so the next rally forms a distinct window.
151         """
152

```

```

153     start_frame: int
154     n_frames: int
155     kind: str = "still" # "still" | "occluded"
156
157
158 @dataclass(frozen=True)
159 class FixtureSpec:
160     """A full synthetic scenario: a frame-indexed sequence of rallies and gaps + its GT.
161
162     ``gts`` are the hand-authored ground-truth rally intervals (seconds) ? the synthetic
163     "labels.csv". For a synthetic fixture they line up with the rallies by construction,
164     which is exactly what makes the quality-floor a *plumbing-correctness* check, not a
165     field-quality one (§6).
166     """
167
168     slug: str
169     description: str
170     fps: float
171     frame_width: float
172     frame_height: float
173     segments: Tuple[Any, ...] # ordered (RallySpec | GapSpec)
174     gts: Tuple[Interval, ...]
175     net_midline: float = 640.0
176
177
178     def _rally_points(spec: RallySpec, net_midline: float, frame_height: float
179                       ) -> List[Tuple[int, int, float, float]]:
180         """Deterministic in-flight shuttle: sine oscillation across the net midline (play
axis = x)."""
181         rows: List[Tuple[int, int, float, float]] = []
182         y_mid = frame_height / 2.0
183         for i in range(spec.n_frames):
184             x = net_midline + spec.amplitude * math.sin(2.0 * math.pi * i / spec.period)
185             # A small orthogonal wobble keeps y-spread < x-spread so the play axis is
unambiguously x.
186             y = y_mid + 30.0 * math.sin(2.0 * math.pi * i / (spec.period / 2.0))
187             rows.append((spec.start_frame + i, 1, x, y))
188         return rows
189
190
191     def _gap_points(spec: GapSpec, net_midline: float, frame_height: float
192                    ) -> List[Tuple[int, int, float, float]]:
193         """Dead-time rows: resting-visible (still) or absent (occluded)."""
194         rows: List[Tuple[int, int, float, float]] = []
195         y_mid = frame_height / 2.0
196         for i in range(spec.n_frames):
197             if spec.kind == "occluded":
198                 rows.append((spec.start_frame + i, 0, 0.0, 0.0))
199             elif spec.kind == "still":
200                 rows.append((spec.start_frame + i, 1, net_midline, y_mid))
201             else: # pragma: no cover - guarded by builder
202                 raise ValueError(f"unknown gap kind {spec.kind!r} (use 'still'|'occluded')")
203         return rows
204
205
206     def make_synthetic_trajectory(spec: FixtureSpec) -> List[Tuple[int, int, float, float]]:
207         """Build the deterministic ``(Frame, Visibility, X, Y)`` rows for a synthetic
scenario."""
208         rows: List[Tuple[int, int, float, float]] = []
209         for seg in spec.segments:
210             if isinstance(seg, RallySpec):
211                 rows.extend(_rally_points(seg, spec.net_midline, spec.frame_height))
212             elif isinstance(seg, GapSpec):
213                 rows.extend(_gap_points(seg, spec.net_midline, spec.frame_height))
214             else: # pragma: no cover - guarded by dataclass shape

```

```

215         raise TypeError(f"unknown segment type {type(seg).__name__}")
216     rows.sort(key=lambda r: r[0])
217     return rows
218
219
220 def write_trajectory_csv(rows: Sequence[Tuple[int, int, float, float]], path: Path) ->
None:
221     """Write trajectory rows as the TrackNet CSV (`Frame,Visibility,X,Y`), LF + 6dp
floats."""
222     lines = ["Frame,Visibility,X,Y"]
223     for frame, vis, x, y in rows:
224         lines.append(f"{int(frame)},{int(vis)},{_fmt(x)},{_fmt(y)}")
225     path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
226
227
228 def write_labels_csv(gts: Sequence[Interval], path: Path) -> None:
229     """Write GT intervals as a `start,end` labels CSV (the strict gt_loader format),
LF."""
230     lines = ["start,end"]
231     for s, e in gts:
232         lines.append(f"{_fmt(s)},{_fmt(e)}")
233     path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
234
235
236 # =====
237 # Built-in synthetic scenarios (the 3 committed slugs)
238 # =====
239
240
241 def _fr(seconds: float, fps: float) -> int:
242     return int(round(seconds * fps))
243
244
245 def builtin_specs() -> List[FixtureSpec]:
246     """The committed synthetic scenarios. Deterministic; chosen so each exercises a
distinct
247     windowing behaviour (one window / dead-time split / occlusion split)."""
248     fps = 30.0
249     fw, fh = 1280.0, 720.0
250     net = 640.0
251
252     # 1) clean_single_rally: one ~4s in-flight rally ? exactly 1 candidate window,
crossings>0.
253     single = FixtureSpec(
254         slug="clean_single_rally",
255         description="One continuous ~4s rally; shuttle oscillates across the net ? 1
window.",
256         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
257         segments=(RallySpec(start_frame=0, n_frames=120),),
258         # Window is [0.033, 3.967]; GT is the rally span the synthetic motion fills.
259         gts=((0.0, 4.0),),
260     )
261
262     # 2) multi_rally_with_deadtime: rally, 3s STILL dead-time, rally ? 2 windows split
by the
263     # low-velocity gap (> merge_gap=2.0s) ? proves dead-time separates windows.
264     r0 = RallySpec(start_frame=0, n_frames=120)
265     gap_still = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="still")
266     r1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
267     multi = FixtureSpec(
268         slug="multi_rally_with_deadtime",
269         description="Two ~4s rallies separated by 3s of still (resting-shuttle) dead-
time ? 2 windows.",
270         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
271         segments=(r0, gap_still, r1),

```

```

272         gts=((0.0, 4.0), (7.0, 11.0)),
273     )
274
275     # 3) occlusion_break: rally, 3s OCCLUDED (Visibility=0), rally ? 2 windows split by
the
276     #     visibility break ? proves occlusion (not just low velocity) separates windows.
277     o0 = RallySpec(start_frame=0, n_frames=120)
278     gap_occ = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="occluded")
279     o1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
280     occ = FixtureSpec(
281         slug="occlusion_break",
282         description="Two ~4s rallies separated by 3s of shuttle occlusion (Visibility=0)
? 2 windows.",
283         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
284         segments=(o0, gap_occ, o1),
285         gts=((0.0, 4.0), (7.0, 11.0)),
286     )
287
288     return [single, multi, occ]
289
290
291     def _spec_by_slug() -> Dict[str, FixtureSpec]:
292         return {s.slug: s for s in builtin_specs()}
293
294
295     # =====
296     # The shared replay (CONTROL / pure-stdlib path)
297     # =====
298
299
300     def _fmt(x: float) -> str:
301         """Render a float with up to 6dp, no scientific notation, ``-0.0`` normalised to
``0.0``."""
302         r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
303         if r == 0.0:
304             r = 0.0
305         return f"{r:.6f}"
306
307
308     def _round(x: float) -> float:
309         r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
310         return 0.0 if r == 0.0 else r
311
312
313     def _pinned_proposal(prov: Optional[Dict[str, Any]]) -> Tuple[float, float, float]:
314         """The windowing proposal tuple ``(vt, mg, mwd)``, read from the fixture provenance
when present
315         (the pinned params travel with the fixture), else the module-level pinned defaults.
NEVER
316         ``windowing.SERVED``. NOTE: the provenance ``max_win`` (W1) is intentionally NOT
read here ?
317         ``build_candidates`` takes only this 3-tuple, so the replay characterizes the W1-OFF
path; the
318         pinned ``max_win`` is a recorded annotation, not a knob the gate applies (see
PINNED_MAX_WINDOW)."""
319         if prov and "windowing" in prov:
320             w = prov["windowing"]
321             return (float(w["vt"]), float(w["mg"]), float(w["mwd"]))
322         return (PINNED_VT, PINNED_MERGE_GAP, PINNED_MIN_WINDOW)
323
324
325     def replay_fixture(slug_or_dir: Any) -> Dict[str, Any]:
326         """Replay one fixture through the shipped CONTROL path; return its recomputed
snapshot.
327

```

```

328     Resolves ``slug_or_dir`` (a slug string or a fixture directory ``Path``) to its
committed
329     ``trajectory.csv`` + ``labels.csv`` + ``provenance.json``, then:
330
331     parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5
shuttle
332     cols only) ? predict(CONTROL, vc) ? score(preds, gts, iou).
333
334     Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
335     control_segments:[[s,e]], score:{...}, gt_hash}``. CONTROL keeps every candidate
then
336     merges overlaps; NO numpy is touched.
337     ""
338     fdir = _resolve_dir(slug_or_dir)
339     slug = fdir.name
340     traj_csv = fdir / "trajectory.csv"
341     labels_csv = fdir / "labels.csv"
342     prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file()
else None
343
344     fps, fw = _fps_fw(prov, slug)
345     iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
346     proposal = _pinned_proposal(prov)
347
348     intervals, feature_rows = distill_local.build_candidates(
349         str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)
350
351     # Build VideoCandidates DIRECTLY with ONLY the 5 shuttle feature columns. We never
call
352     # fusion_compare.load_videos / ablation.load_videos ? those read c['person']
unconditionally
353     # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
354     features: Dict[str, List[float]] = {
355         name: [row[i] for row in feature_rows] for i, name in
enumerate(SHUTTLE_FEATURES)
356     }
357     gts = load_gt_intervals(str(labels_csv), strict=True)
358     vc = VideoCandidates(name=slug, intervals=intervals, features=features, gts=gts)
359
360     preds = predict(CONTROL, vc) # CONTROL: keep all ? merge_overlaps; pure-stdlib, no
numpy
361     sc = score(preds, gts, iou)
362
363     candidates: List[Dict[str, Any]] = []
364     for j, (s, e) in enumerate(intervals):
365         shuttle = {name: _round(feature_rows[j][i]) for i, name in
enumerate(SHUTTLE_FEATURES)}
366         # net_crossings is conceptually an integer count ? keep it exact (int), per
constraint 2.
367         shuttle["net_crossings"] =
int(round(feature_rows[j][SHUTTLE_FEATURES.index("net_crossings")]))
368         candidates.append({"start": _round(s), "end": _round(e), "shuttle": shuttle})
369
370     return {
371         "slug": slug,
372         "schema_version": CURRENT_SCHEMA,
373         "iou": _round(iou),
374         "candidates": candidates,
375         "control_segments": [[_round(s), _round(e)] for s, e in preds],
376         "score": {
377             "iou_threshold": _round(sc.iou_threshold),
378             "num_pred": int(sc.num_pred),
379             "num_gt": int(sc.num_gt),
380             "true_positives": int(sc.true_positives),
381             "false_positives": int(sc.false_positives),

```

```

382         "false_negatives": int(sc.false_negatives),
383         "precision": _round(sc.precision),
384         "recall": _round(sc.recall),
385         "f1": _round(sc.f1),
386         "mean_iou": _round(sc.mean_iou),
387         "mean_start_error": _round(sc.mean_start_error),
388         "mean_end_error": _round(sc.mean_end_error),
389     },
390     "gt_hash": gt_hash(gts),
391 }
392
393
394 # =====
395 # Freeze / refresh (expected.json + provenance.json are correct-by-construction)
396 # =====
397
398
399 def _provenance(spec_or_dir: Any, slug: str, fps: float, fw: float, iou: float = 0.5
400                ) -> Dict[str, Any]:
401     """The committed provenance block ? pins the windowing + privacy/licensing tags
402     (§4c, §5)."""
403     desc = ""
404     if isinstance(spec_or_dir, FixtureSpec):
405         desc = spec_or_dir.description
406     return {
407         "slug": slug,
408         "schema_version": CURRENT_SCHEMA,
409         "kind": "synthetic",
410         "license": "synthetic",
411         "minors_free": True,
412         "description": desc,
413         "fps": _round(fps),
414         "frame_width": _round(fw),
415         "iou": _round(iou),
416         "windowing": {
417             "vt": PINNED_VT,
418             "mg": PINNED_MERGE_GAP,
419             "mwd": PINNED_MIN_WINDOW,
420             "max_win": PINNED_MAX_WINDOW,
421         },
422         "paths": {
423             "trajectory": "trajectory.csv",
424             "labels": "labels.csv",
425             "expected": "expected.json",
426         },
427         "note": (
428             "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. "
429             "Plumbing-correctness fixture ? NOT a measure of rally-detection quality
430 (§6).",
431     ),
432 }
433
434 def freeze_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR,
435                   iou: float = 0.5) -> Dict[str, Any]:
436     """Compute ``expected.json`` + ``provenance.json`` for one fixture from its
437     COMMITTED
438     ``trajectory.csv`` + ``labels.csv`` via the replay (so expected is correct-by-
439     construction).
440
441     Idempotent: rewrites the same bytes given the same committed inputs. Requires the
442     trajectory
443     + labels CSVs to already exist (use ``--freeze-synthetic`` to generate them first).
444     """
445     fdir = fixtures_dir / slug

```

```

442     traj_csv, labels_csv = fdir / "trajectory.csv", fdir / "labels.csv"
443     if not traj_csv.is_file() or not labels_csv.is_file():
444         raise FileNotFoundError(
445             f"[slug]: trajectory.csv/labels.csv missing ? run --freeze-synthetic to
generate them")
446
447     # Provenance first (it pins the windowing the replay reads), then replay ? expected.
448     spec = _spec_by_slug().get(slug)
449     fps, fw = (spec.fps, spec.frame_width) if spec is not None else
_fps_fw_from_prov(fdir, slug)
450     prov = _provenance(spec if spec is not None else fdir, slug, fps, fw, iou)
451     _write_json(fdir / "provenance.json", prov)
452
453     expected = replay_fixture(fdir)
454     _write_json(fdir / "expected.json", expected)
455     return expected
456
457
458     def freeze_synthetic(fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
459         """Generate trajectory.csv + labels.csv for every built-in scenario, freeze
expected.json +
460         provenance.json, and (re)write the manifest. Returns the slugs frozen."""
461         specs = builtin_specs()
462         fixtures_dir.mkdir(parents=True, exist_ok=True)
463         slugs: List[str] = []
464         for spec in specs:
465             fdir = fixtures_dir / spec.slug
466             fdir.mkdir(parents=True, exist_ok=True)
467             rows = make_synthetic_trajectory(spec)
468             write_trajectory_csv(rows, fdir / "trajectory.csv")
469             write_labels_csv(spec.gts, fdir / "labels.csv")
470             freeze_fixture(spec.slug, fixtures_dir=fixtures_dir)
471             slugs.append(spec.slug)
472         _write_manifest(specs, fixtures_dir)
473         return slugs
474
475
476     def refresh_snapshot(slug: Optional[str] = None, all_slugs: bool = False,
477                         fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
478         """Re-freeze expected.json (+ provenance.json) from the COMMITTED trajectory.csv +
labels.csv ?
479         NO regeneration of the trajectory/labels. ? $6: this can rubber-stamp a real
regression; the
480         mitigation is reviewing the visible diff + recording a reason (the CLI requires
one)."""
481         if all_slugs:
482             targets = [d.name for d in sorted(fixtures_dir.iterdir())
483                        if d.is_dir() and (d / "trajectory.csv").is_file()]
484         elif slug:
485             targets = [slug]
486         else:
487             raise ValueError("refresh_snapshot needs --slug S or --all")
488         for s in targets:
489             freeze_fixture(s, fixtures_dir=fixtures_dir)
490         return targets
491
492
493     # =====
494     # P1 ? De-attribution + minimization: OWNED real trajectory+labels ? committable Tier-0
495     # fixture, reusing the SAME replay_fixture snapshot. (docs/GOLDEN_REGRESSION_FIXTURES.md
496     # $4c anti-attribution, $5 threat model, $7 row P1, $6 honest limits.)
497     # =====
498
499
500     class RefusalError(ValueError):

```

```

501         """The provenance/biometric refusal gate (M2) rejected an unsafe freeze.
502
503         A subclass of ``ValueError`` (so existing ``except ValueError`` paths still catch
504         it) that
505         names *why* the unsafe public-fixture path was refused. The refusal is the code-
506         enforced
507         provenance gate (§4c "provenance hard-fail gate", §7 ?): the owner-recorded /
508         minors-excluded
509         / license flags are human-asserted, but the gate makes the unsafe path impossible to
510         reach
511         *silently* ? a fixture cannot be written unless all three are affirmatively set.
512         """
513
514         def _new_surrogate_slug() -> str:
515             """An opaque RANDOM surrogate slug ? ``fx_<16 hex>`` from ``os.urandom`` (§4c).
516             Deliberately NOT the MD5 ``video_id`` and NOT ``HMAC(salt, video_id?name)`` ? a
517             random,
518             NON-reversible id with no algebraic link to the source, so it can't be brute-forced
519             over a
520             tiny known roster even if a salt leaked. The surrogate?source map (if kept) lives
521             off-git.
522             """
523             return f"fx_{os.urandom(8).hex()}"
524
525         #: The fixed schema filenames the writers ALWAYS emit (as ``paths`` literals). A source-
526         path token
527         #: that EXACTLY equals one of these is NOT a leak (the bytes would be written
528         regardless), so it is
529         #: excluded. Matched by exact membership, NEVER substring containment ? a substring rule
530         would let
531         #: a real source named ``label.csv`` (stem ``label`` ? ``labels.csv``) leak unnoticed
532         (red-team #4).
533         _SCHEMA_FILENAME_LITERALS: Tuple[str, ...] = (
534             "trajectory", "trajectory.csv", "labels", "labels.csv", "expected", "expected.json",
535             "provenance", "provenance.json",
536         )
537
538         def _scan_forbidden(text: str, *, where: str, source_paths: Sequence[str]) -> None:
539             """POSITIVE privacy assertion (§4c, red-team ?): raise if ``text`` leaks anything
540             attributable.
541
542             Scans the written JSON *text* for (a) any ``FORBIDDEN_KEYS`` biometric/identity
543             substring,
544             (b) a 32-hex MD5 (the ``video_id`` shape), or (c) any identifying token of a source
545             path
546             (full path / basename / stem) ? excluding the fixed schema filename literals the
547             writers
548             always emit. The trajectory is shuttle-only by nature; this guards against a
549             *regression*
550             re-introducing a person/audio/pose stream or an id/path back-channel ? never the
551             silent default.
552             """
553             low = text.lower()
554             for key in FORBIDDEN_KEYS:
555                 if key in low:
556                     raise RefusalError(
557                         f"{where}: forbidden key substring {key!r} found in written output ? "
558                         f"Tier-0 fixtures are shuttle-only; person/pose/gait/face/audio/player
559                         MUST NOT "
560                         f"appear (docs/GOLDEN_REGRESSION_FIXTURES.md §4c).")
561             m = _MD5_RE.search(text)

```

```

548         if m:
549             raise RefusalError(
550                 f"{where}: a 32-hex MD5 ({m.group(0)!r}) found in written output ? that is
the "
551                 f"video_id shape and an attribution back-channel; never write it ($4c/$5).")
552     for sp in source_paths:
553         if not sp:
554             continue
555         p = Path(sp)
556         for token in (sp, p.name, p.stem):
557             t = (token or "").strip().lower()
558             # Skip ONLY a token that EXACTLY equals a schema-literal the writers always
emit
559             # (e.g. a source named ``trajectory.csv``) ? that collision is not a leak.
We must NOT
560             # skip on substring containment: a token like ``label`` or ``traj`` is a
SUBSTRING of
561             # ``labels.csv``/``trajectory.csv``, so substring-skip would silently miss a
real source
562             # named ``label.csv`` leaking its stem (red-team #4). Exact membership only.
563             if not t or t in _SCHEMA_FILENAME_LITERALS:
564                 continue
565             if t in low:
566                 raise RefusalError(
567                     f"{where}: source-path token {token!r} found in written output ? the
source "
568                     f"filename/path must never be persisted ($4c metadata strip).")
569
570
571     def _reset_trajectory_rows(points: Sequence[Any], t0_frame: int
572                               ) -> List[Tuple[int, int, float, float]]:
573         """Shift every ``TrajectoryPoint`` frame by ``-t0_frame`` ?
``(Frame,Visibility,X,Y)`` rows.
574
575         Visibility/X/Y are passed through untouched (the shuttle geometry is metric-
invariant under a
576         pure time shift); only the absolute frame index moves toward 0, severing the match
timeline.
577         """
578         rows: List[Tuple[int, int, float, float]] = []
579         for p in points:
580             rows.append((int(p.frame) - t0_frame, 1 if p.visible else 0, float(p.x),
float(p.y)))
581         rows.sort(key=lambda r: r[0])
582         return rows
583
584
585     def _reset_labels(gts: Sequence[Interval], t0_sec: float) -> List[Interval]:
586         """Shift every label by ``-t0_sec`` (same offset as the trajectory ? metric-
invariant).
587
588         ``t0_sec`` MUST already be quantized to ``_FLOAT_ROUND_DP`` decimals by the caller.
Each label
589         endpoint is ALSO rounded to 6dp BEFORE subtracting, so the shift is the difference
of two exact
590         6dp quantities ? no double-rounding (``round6(orig - round6(t0))`` vs the no-reset
591         ``round6(orig)`` accumulated error; red-team #2). This keeps the RAW reset-vs-no-
reset score
592         delta at the float-error floor (~3.3e-7, well within ``_FLOAT_TOL``); see
``freeze_real_fixture``
593         for why the SERIALIZED 6dp boundary error can still differ by one 6dp ULP on a sub-
second
594         offset (the grid, not a real metric divergence)."""
595         return [(_round(s) - t0_sec, _round(e) - t0_sec) for s, e in gts]
596

```

```

597
598     def freeze_real_fixture(
599         label: str,
600         trajectory_csv_path: Any,
601         labels_csv_path: Any,
602         *,
603         license: str,
604         minors_free: bool,
605         owned: bool,
606         fps: float,
607         frame_width: float,
608         fixtures_dir: Path = FIXTURES_DIR,
609         slug: Optional[str] = None,
610         time_origin_reset: bool = True,
611         source_note: str = "",
612     ) -> Dict[str, Any]:
613         """Turn an OWNED, minors-free real trajectory + golden labels into a committable,
de-attributed
614         Tier-0 fixture ? reusing M1's ``replay_fixture`` for the snapshot. ($4c / $5 / $7
row P1.)
615
616         The refusal gate (M2) makes the unsafe path impossible to reach silently: a fixture
is emitted
617         ONLY when ``minors_free is True`` AND ``owned is True`` AND ``license`` is a non-
blank string,
618         AND ``label`` is a strict slug (lowercase alnum + ``_``/``-`` ? a free-text
identifier like a
619         player/event name is refused, red-team #1). The committed bytes carry NO filename /
video_id /
620         match / player / capture-date / operator free-text ? only an opaque random surrogate
slug, the
621         shuttle-only trajectory (time-origin reset by default), the reset labels, and a
622         ``kind="cleared_real"`` provenance block. A positive privacy assertion re-reads the
written
623         ``expected.json`` + ``provenance.json`` and refuses if anything attributable slipped
in.
624
625         ``source_note`` is **accepted but NOT persisted** ? operator free-text is an
attribution
626         back-channel ``_scan_forbidden`` cannot fully police (an identifying note that
happens to
627         contain no FORBIDDEN_KEY / MD5 / source-path token would slip through), so it never
reaches the
628         committed ``provenance.json`` (red-team #1). It is still scanned defensively and the
flag is
629         retained for CLI compatibility / future logging.
630
631         NOTE ($6 honest limits): the random surrogate + reset timeline make the fixture non-
attributable
632         only *relative to the unpublished source* ? it is NOT absolutely anonymous (EDPS v.
SRB). De-
633         attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off
(P2) and the
634         owner-asserted Fork-A / minors / biometric-exclusion flags.
635
636         The fixture is written into a temp dir and promoted (``os.replace``) to the final
slug dir ONLY
637         after the privacy scan passes ? a refused / failed freeze leaves NO partial fixture
dir
638         (red-team #3).
639
640         Returns the recomputed ``expected.json`` snapshot dict (identical to
``replay_fixture``).
641         """
642         # --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail ($4c / $7 ?)

```

```

-----
643     reasons: List[str] = []
644     if minors_free is not True:
645         reasons.append("minors_free must be True (DPDP child = under 18; behavioural
monitoring "
646                         "of a minor is prohibited ? §3, guardrail 'exclude minors')")
647     if owned is not True:
648         reasons.append("owned must be True (only owner-recorded Fork-A source is safe to
commit; "
649                         "broadcast/scraped = Fork B, do NOT commit ? §3 D1)")
650     if not (isinstance(license, str) and license.strip()):
651         reasons.append("license must be a non-blank string (per-record provenance tag ?
§4c "
652                         "'provenance hard-fail gate')")
653     # label is committed verbatim into provenance ? it MUST be a strict slug, never
operator free
654     # text (a name/event), which _scan_forbidden cannot reliably catch (red-team #1
free-text leak).
655     if not (isinstance(label, str) and _LABEL_SLUG_RE.match(label)):
656         reasons.append(
657             f"label must match the strict slug charset {_LABEL_SLUG_RE.pattern!r}
(lowercase "
658             f"alnum + '_'/'-', no spaces/punctuation) ? a free-text identifier
(player/event "
659             f"name) is an attribution leak; got {label!r} (§4c metadata strip, red-team
#1)")
660     if reasons:
661         raise RefusalError(
662             "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution
does NOT "
663             "cure provenance/licensing; this gate sits behind counsel sign-off,
§6/P2):\n - "
664             + "\n - ".join(reasons))
665
666     traj_path = Path(str(trajjectory_csv_path))
667     labels_path = Path(str(labels_csv_path))
668
669     # --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source
name ---
670     out_slug = slug if slug is not None else _new_surrogate_slug()
671
672     # --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient)
----
673     points = TrackNetRunner.parse_trajectory_csv(str(traj_path))
674     if not points:
675         raise ValueError(f"{traj_path}: no parseable trajectory rows
(Frame,Visibility,X,Y)")
676     gts = load_gt_intervals(str(labels_path), strict=True)
677
678     # --- (d) consistent time-origin reset (default ON; metric-invariant within
_FLOAT_TOL, §4c) --
679     # Subtract the SAME offset from BOTH the trajectory frames and the label seconds:
metrics.score
680     # is built from pure differences (interval_iou, |start-start|, |end-end|), so a pure
time shift
681     # leaves the score essentially unchanged ? only the absolute timeline is severed. To
keep the
682     # two shifts a consistent quantity (and avoid double-rounding past _FLOAT_TOL ? red-
team #2),
683     # t0_sec is ROUNDED to 6dp BEFORE subtracting (and _reset_labels rounds each
endpoint first), so
684     # the RAW reset-vs-no-reset score delta sits at the float-error floor (~3.3e-7 ?
_FLOAT_TOL).
685     #
686     # HONEST LIMIT (not a bug): the trajectory shifts by an INTEGER frame count while

```

```

labels shift
687      # in SECONDS, and 6dp-rounding is NOT additive (round6(8/fps) - round6(7/fps) ?
round6(1/fps)),
688      # so the *serialized* 6dp boundary error (mean_start/end_error) can differ by ONE
6dp ULP
689      # (1e-6) on a sub-second offset. That is the serialization grid, not a metric
divergence ? the
690      # score is bit-identical for whole-second offsets and within _FLOAT_TOL otherwise
(test #2).
691      if time_origin_reset:
692          t0_frame = min(int(p.frame) for p in points)
693          t0_sec = _round(t0_frame / float(fps))
694      else:
695          t0_frame, t0_sec = 0, 0.0
696      rows = _reset_trajectory_rows(points, t0_frame)
697      reset_gts = _reset_labels(gts, t0_sec)
698
699      # --- (d2) negative-label guard BEFORE any write (red-team #3a)
-----
700      # A reset label that starts < 0 means the label timeline predates the earliest
trajectory
701      # frame ? an inconsistent input that would crash the strict gt_loader on replay
(negative
702      # start). Refuse loudly up-front so the partial-write path is never even reached.
703      neg = [(s, e) for (s, e) in reset_gts if s < 0]
704      if neg:
705          raise RefusalError(
706              f"REFUSED: label timeline predates the trajectory ? {len(neg)} reset
label(s) have a "
707              f"negative start after the t0={t0_sec:.6f}s time-origin reset (e.g.
{neg[0]!r}). The "
708              f"earliest label must not precede the earliest trajectory frame ($4c time-
origin reset).")
709
710      iou = 0.5
711      # NOTE (red-team #1b): operator free-text `source_note` is DELIBERATELY NOT a key in
`prov` ?
712      # it never reaches the committed bytes (an identifying note is an attribution back-
channel
713      # _scan_forbidden cannot fully police). The fixed boilerplate `note` constant below
stays.
714      prov = {
715          "slug": out_slug,
716          "schema_version": CURRENT_SCHEMA,
717          "kind": "cleared_real",
718          "label": str(label),
719          "license": license,
720          "minors_free": True,
721          "owned": True,
722          "fps": _round(fps),
723          "frame_width": _round(frame_width),
724          "iou": _round(iou),
725          "time_origin_reset": bool(time_origin_reset),
726          "windowing": {
727              "vt": PINNED_VT,
728              "mg": PINNED_MERGE_GAP,
729              "mwd": PINNED_MIN_WINDOW,
730              "max_win": PINNED_MAX_WINDOW,
731          },
732          "paths": {
733              "trajectory": "trajectory.csv",
734              "labels": "labels.csv",
735              "expected": "expected.json",
736          },
737          "gt_hash": gt_hash(reset_gts),

```

```

738         "note": (
739             "De-attributed REAL Fork-A fixture (owner-recorded, owned, minors-excluded,
biometric-"
740             "free, shuttle-only). Opaque RANDOM surrogate slug; absolute timeline reset.
NON-"
741             "attributable only relative to the UNPUBLISHED source (EDPS v. SRB) ? never
'anonymous'. "
742             "Tier-0 / CONTROL-only. $6: behaviour-locking, not correctness; sits behind
counsel (P2).",
743         ),
744     }
745
746     # --- (e) write into a TEMP dir, replay, scan, and only THEN promote (red-team #3b)
-----
747     # Everything is written under a sibling temp dir; the final slug dir is created by
an atomic
748     # os.replace ONLY after the privacy scan passes. A refused/failed freeze therefore
leaves NO
749     # partial fixture dir behind. source_note (if any) is scanned defensively but never
written.
750     fdir = fixtures_dir / out_slug
751     fixtures_dir.mkdir(parents=True, exist_ok=True)
752     if fdir.exists():
753         raise ValueError(f"{fdir}: fixture dir already exists ? refuse to clobber
(choose a new slug)")
754     tmp_parent = Path(tempfile.mkdtemp(prefix=f"{out_slug}.tmp.",
dir=str(fixtures_dir)))
755     tmp_dir = tmp_parent / out_slug
756     tmp_dir.mkdir(parents=True, exist_ok=True)
757     try:
758         write_trajectory_csv(rows, tmp_dir / "trajectory.csv")
759         write_labels_csv(reset_gts, tmp_dir / "labels.csv")
760         _write_json(tmp_dir / "provenance.json", prov)
761
762         expected = replay_fixture(tmp_dir) # REUSE M1 ? correct-by-construction
snapshot
763         _write_json(tmp_dir / "expected.json", expected)
764
765         # --- (f) POSITIVE privacy assertion BEFORE promotion (scan written text +
source_note) ---
766         source_paths = [str(traj_path), str(labels_path)]
767         _scan_forbidden((tmp_dir / "expected.json").read_text(encoding="utf-8"),
768                        where=f"{out_slug}/expected.json", source_paths=source_paths)
769         _scan_forbidden((tmp_dir / "provenance.json").read_text(encoding="utf-8"),
770                        where=f"{out_slug}/provenance.json", source_paths=source_paths)
771         # source_note is never persisted, but a leak in it is still a contract violation
? scan it.
772         if source_note:
773             _scan_forbidden(str(source_note),
774                            where=f"{out_slug}/source_note(not-persisted)",
source_paths=source_paths)
775
776         # --- (g) PROMOTE atomically: temp slug dir ? final slug dir (scan passed)
-----
777         os.replace(str(tmp_dir), str(fdir))
778     finally:
779         # Clean up the temp parent whether we promoted or refused (a refusal leaves NO
partial dir).
780         shutil.rmtree(str(tmp_parent), ignore_errors=True)
781
782     # --- (h) append to the manifest with kind="cleared_real" (don't clobber synthetic)
-----
783     _append_manifest_entry(prov, fixtures_dir)
784     return expected
785

```

```

786
787 def _manifest_entry_for(prov: Dict[str, Any]) -> Dict[str, Any]:
788     """A manifest row for a single fixture, derived from its provenance block."""
789     slug = prov["slug"]
790     return {
791         "slug": slug,
792         "fps": _round(float(prov["fps"])),
793         "frame_width": _round(float(prov["frame_width"])),
794         "windowing": dict(prov.get("windowing", {})),
795         "kind": prov.get("kind", "synthetic"),
796         "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
797         "paths": {
798             "dir": slug,
799             "trajectory": f"{slug}/trajectory.csv",
800             "labels": f"{slug}/labels.csv",
801             "expected": f"{slug}/expected.json",
802             "provenance": f"{slug}/provenance.json",
803         },
804     }
805
806
807 def _append_manifest_entry(prov: Dict[str, Any], fixtures_dir: Path) -> None:
808     """Add/replace one fixture's manifest row WITHOUT clobbering the others (e.g.
809     synthetic)."""
810     entries = load_manifest(fixtures_dir)
811     entries = [e for e in entries if e.get("slug") != prov["slug"]]
812     entries.append(_manifest_entry_for(prov))
813     entries.sort(key=lambda e: str(e.get("slug")))
814     _write_json(fixtures_dir / "manifest.json", entries)
815
816 # =====
817 # Manifest + loaders
818 # =====
819
820
821 def _write_manifest(specs: Sequence[FixtureSpec], fixtures_dir: Path) -> None:
822     entries = [{
823         "slug": spec.slug,
824         "fps": _round(spec.fps),
825         "frame_width": _round(spec.frame_width),
826         "windowing": {
827             "vt": PINNED_VT, "mg": PINNED_MERGE_GAP,
828             "mwd": PINNED_MIN_WINDOW, "max_win": PINNED_MAX_WINDOW,
829         },
830         "kind": "synthetic",
831         "schema_version": CURRENT_SCHEMA,
832         "paths": {
833             "dir": spec.slug,
834             "trajectory": f"{spec.slug}/trajectory.csv",
835             "labels": f"{spec.slug}/labels.csv",
836             "expected": f"{spec.slug}/expected.json",
837             "provenance": f"{spec.slug}/provenance.json",
838         },
839     } for spec in specs]
840     _write_json(fixtures_dir / "manifest.json", entries)
841
842
843 def load_manifest(fixtures_dir: Path = FIXTURES_DIR) -> List[Dict[str, Any]]:
844     """Load the fixtures manifest (returns [] if absent)."""
845     path = fixtures_dir / "manifest.json"
846     if not path.is_file():
847         return []
848     data = _read_json(path)
849     if not isinstance(data, list): # pragma: no cover - corrupt manifest

```

```

850         raise ValueError(f"{path}: manifest must be a JSON array")
851     return data
852
853
854     def load_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR) -> Dict[str, Any]:
855         """Load one fixture's committed artifacts: ``{provenance, expected, dir}``. Refuses
a fixture
856         whose ``schema_version`` exceeds ``CURRENT_SCHEMA`` (forward-incompatible) on EITHER
the
857         ``expected`` OR the ``provenance`` block ? they are guarded INDEPENDENTLY so a
forward-version
858         bump in one is never masked by an in-range value in the other (red-team #7)."""
859         fdir = fixtures_dir / slug
860         prov = _read_json(fdir / "provenance.json")
861         expected = _read_json(fdir / "expected.json")
862         for which, blob in (("expected", expected), ("provenance", prov)):
863             sv = int(blob.get("schema_version", 0))
864             if sv > CURRENT_SCHEMA:
865                 raise ValueError(
866                     f"{slug}: {which} schema_version {sv} > CURRENT_SCHEMA {CURRENT_SCHEMA}
? "
867                     f"upgrade the harness")
868         return {"slug": slug, "dir": fdir, "provenance": prov, "expected": expected}
869
870
871     # =====
872     # Parse-compare (NEVER byte-compare): exact ints/structure, abs-tol floats
873     # =====
874
875
876     def compare_expected(recomputed: Dict[str, Any], committed: Dict[str, Any]) ->
List[str]:
877         """Return a list of human-readable mismatch messages (empty == match).
878
879         Parse-compare per HARD CONSTRAINT 2: structural/int fields (candidate count,
net_crossings,
880         TP/FP/FN, segment count, num_pred/num_gt, slug, schema_version, gt_hash) must be
EXACTLY
881         equal; float fields compared with abs tolerance 1e-6. Never a byte compare ?
CRLF/float-format
882         safe (the repo runs with core.autocrlf=true).
883
884         Key-set equality is enforced on BOTH the top-level dict AND the ``score`` block
(mirroring the
885         per-candidate ``shuttle`` check) so an EXTRA key on one side is a mismatch, not a
silent pass ?
886         a fixed-field loop alone would ignore an added field (red-team #5)."""
887         out: List[str] = []
888
889         def exact(path: str, a: Any, b: Any) -> None:
890             if a != b:
891                 out.append(f"{path}: {a!r} != {b!r} (exact)")
892
893         def approx(path: str, a: Any, b: Any) -> None:
894             try:
895                 if abs(float(a) - float(b)) > _FLOAT_TOL:
896                     out.append(f"{path}: {a!r} != {b!r} (|?| > {_FLOAT_TOL})")
897             except (TypeError, ValueError):
898                 out.append(f"{path}: non-numeric float field {a!r} vs {b!r}")
899
900         # Top-level key-set equality ? an extra/missing top-level key is a mismatch (red-
team #5).
901         if set(recomputed) != set(committed):
902             out.append(f"(top-level): keys {sorted(recomputed)} != {sorted(committed)}
(exact)")

```

```

903
904     exact("slug", recomputed.get("slug"), committed.get("slug"))
905     exact("schema_version", recomputed.get("schema_version"),
committed.get("schema_version"))
906     exact("gt_hash", recomputed.get("gt_hash"), committed.get("gt_hash"))
907     approx("iou", recomputed.get("iou"), committed.get("iou"))
908
909     rc, cc = recomputed.get("candidates", []), committed.get("candidates", [])
910     if len(rc) != len(cc):
911         out.append(f"candidates: count {len(rc)} != {len(cc)} (exact)")
912     else:
913         for i, (r, c) in enumerate(zip(rc, cc)):
914             approx(f"candidates[{i}].start", r.get("start"), c.get("start"))
915             approx(f"candidates[{i}].end", r.get("end"), c.get("end"))
916             rs, cs = r.get("shuttle", {}), c.get("shuttle", {})
917             if set(rs) != set(cs):
918                 out.append(f"candidates[{i}].shuttle: keys {sorted(rs)} != {sorted(cs)}
(exact)")
919             else:
920                 for k in rs:
921                     if k == "net_crossings":
922                         exact(f"candidates[{i}].shuttle.net_crossings", rs[k], cs[k])
923                     else:
924                         approx(f"candidates[{i}].shuttle.{k}", rs[k], cs[k])
925
926     rseg, cseg = recomputed.get("control_segments", []),
committed.get("control_segments", [])
927     if len(rseg) != len(cseg):
928         out.append(f"control_segments: count {len(rseg)} != {len(cseg)} (exact)")
929     else:
930         for i, (r, c) in enumerate(zip(rseg, cseg)):
931             approx(f"control_segments[{i}][0]", r[0], c[0])
932             approx(f"control_segments[{i}][1]", r[1], c[1])
933
934     rsc, csc = recomputed.get("score", {}), committed.get("score", {})
935     # Score-block key-set equality ? an extra/missing score key is a mismatch (red-team
#5).
936     if set(rsc) != set(csc):
937         out.append(f"score: keys {sorted(rsc)} != {sorted(csc)} (exact)")
938     int_fields = ("num_pred", "num_gt", "true_positives", "false_positives",
"false_negatives")
939     float_fields = ("iou_threshold", "precision", "recall", "f1", "mean_iou",
940                     "mean_start_error", "mean_end_error")
941     for k in int_fields:
942         exact(f"score.{k}", rsc.get(k), csc.get(k))
943     for k in float_fields:
944         approx(f"score.{k}", rsc.get(k), csc.get(k))
945
946     return out
947
948
949 # =====
950 # Small JSON / path helpers (deterministic writers: sort_keys, LF, trailing newline)
951 # =====
952
953
954 def _write_json(path: Path, obj: Any) -> None:
955     text = json.dumps(obj, indent=2, sort_keys=True, ensure_ascii=False)
956     path.write_text(text + "\n", encoding="utf-8", newline="\n")
957
958
959 def _read_json(path: Path) -> Any:
960     with open(path, encoding="utf-8") as f:
961         return json.load(f)
962

```

```

963
964 def _resolve_dir(slug_or_dir: Any) -> Path:
965     if isinstance(slug_or_dir, Path):
966         return slug_or_dir
967     p = Path(str(slug_or_dir))
968     if p.is_dir():
969         return p
970     return FIXTURES_DIR / str(slug_or_dir)
971
972
973 def _fps_fw(prov: Optional[Dict[str, Any]], slug: str) -> Tuple[float, float]:
974     if prov and "fps" in prov and "frame_width" in prov:
975         return float(prov["fps"]), float(prov["frame_width"])
976     spec = _spec_by_slug().get(slug)
977     if spec is not None:
978         return spec.fps, spec.frame_width
979     raise ValueError(f"{slug}: no fps/frame_width in provenance and not a built-in
spec")
980
981
982 def _fps_fw_from_prov(fdir: Path, slug: str) -> Tuple[float, float]:
983     prov_path = fdir / "provenance.json"
984     prov = _read_json(prov_path) if prov_path.is_file() else None
985     return _fps_fw(prov, slug)
986
987
988 # =====
989 # CLI
990 # =====
991
992
993 def _cmd_check(fixtures_dir: Path) -> int:
994     manifest = load_manifest(fixtures_dir)
995     if not manifest:
996         print("[golden-fixtures] no fixtures found ? run --freeze-synthetic first",
file=sys.stderr)
997         return 1
998     failures = 0
999     for entry in manifest:
1000         slug = entry["slug"]
1001         committed = _read_json(fixtures_dir / slug / "expected.json")
1002         recomputed = replay_fixture(fixtures_dir / slug)
1003         mismatches = compare_expected(recomputed, committed)
1004         if mismatches:
1005             failures += 1
1006             print(f"[FAIL] {slug}: {len(mismatches)} mismatch(es) "
f"(characterization = behaviour-locking, NOT correctness ? $6):")
1007             for m in mismatches:
1008                 print(f"    - {m}")
1009         else:
1010             sc = recomputed["score"]
1011             print(f"[ok] {slug}: {len(recomputed['candidates'])} candidates, "
f"{len(recomputed['control_segments'])} segments, F1={sc['f1']:.6f}")
1012     print(f"[golden-fixtures] {len(manifest) - failures}/{len(manifest)} fixtures match
"
1013
1014     f"(synthetic-tier = plumbing correctness, not field quality ? $6)")
1015     return 0 if failures == 0 else 1
1016
1017
1018
1019 def main(argv: Optional[Sequence[str]] = None) -> int:
1020     ap = argparse.ArgumentParser(
1021         description="Golden Regression Fixtures (M1) ? synthetic, CONTROL-only replay
harness.")
1022     g = ap.add_mutually_exclusive_group(required=True)
1023     g.add_argument("--freeze-synthetic", action="store_true",

```

```

1024             help="generate synthetic trajectory+labels for all built-in
scenarios, then "
1025             "freeze expected.json+provenance.json + write manifest.json")
1026         g.add_argument("--refresh-snapshot", action="store_true",
1027             help="re-freeze expected.json from COMMITTED trajectory+labels (no
regeneration)")
1028         g.add_argument("--check", action="store_true",
1029             help="replay all fixtures + parse-compare vs committed expected.json
(exit 0/1)")
1030         g.add_argument("--freeze-real", action="store_true",
1031             help="(P1) de-attribute an OWNED real trajectory+labels into a
committable Tier-0 "
1032             "fixture; REFUSED without --owned --minors-free and a non-blank
--license")
1033         ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate
slug for "
1034             "--freeze-real (default: a random fx<hex> id)")
1035         ap.add_argument("--all", action="store_true", help="all slugs for --refresh-
snapshot")
1036         ap.add_argument("--reason", help="REQUIRED recorded reason for --refresh-snapshot
(review the diff!)")
1037         # --freeze-real (P1 / M2 refusal gate) options:
1038         ap.add_argument("--trajectory", help="--freeze-real) source trajectory CSV
(Frame,Visibility,X,Y)")
1039         ap.add_argument("--labels", help="--freeze-real) source golden labels CSV
(start,end)")
1040         ap.add_argument("--license", dest="license_", default=None,
1041             help="--freeze-real) the source license tag (required, non-blank)")
1042         ap.add_argument("--owned", action="store_true",
1043             help="--freeze-real) assert owner-recorded Fork-A source
(required)")
1044         ap.add_argument("--minors-free", dest="minors_free", action="store_true",
1045             help="--freeze-real) assert the source contains NO minors
(required)")
1046         ap.add_argument("--fps", type=float, help="--freeze-real) source fps")
1047         ap.add_argument("--frame-width", dest="frame_width", type=float, help="--freeze-
real) source frame width (px)")
1048         ap.add_argument("--label", default="cleared_real",
1049             help="--freeze-real) non-identifying scenario label ? STRICT slug
only "
1050             "(lowercase alnum + '_'/'-', no spaces/punctuation/names)")
1051         ap.add_argument("--source-note", dest="source_note", default="",
1052             help="--freeze-real) ACCEPTED BUT NOT PERSISTED ? operator free-
text is an "
1053             "attribution back-channel, so it never reaches the committed
provenance "
1054             "(scanned defensively, then dropped). Retained for CLI
compatibility.")
1055         ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
action="store_false",
1056             help="--freeze-real) DISABLE the anti-attribution time-origin reset
(default ON)")
1057         ap.set_defaults(time_origin_reset=True)
1058         ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
1059             help="override the fixtures directory (default: repo
eval_baselines/fixtures)")
1060         args = ap.parse_args(list(argv) if argv is not None else None)
1061
1062         fixtures_dir: Path = args.fixtures_dir
1063
1064         if args.freeze_synthetic:
1065             slugs = freeze_synthetic(fixtures_dir)
1066             print(f"[golden-fixtures] froze {len(slugs)} synthetic fixture(s): {'',
'.join(slugs)}")
1067             print(f" -> {fixtures_dir}")

```

```

1068         return 0
1069
1070     if args.refresh_snapshot:
1071         if not args.slug and not args.all:
1072             ap.error("--refresh-snapshot requires --slug S or --all")
1073         # --reason is MANDATORY (red-team #6): --refresh-snapshot can rubber-stamp a
1074         real
1075         # regression, so the deliberate, recorded reason is a required arg, not a soft
1076         warning.
1077         if not (args.reason and args.reason.strip()):
1078             ap.error("--refresh-snapshot requires --reason \"<why>\" ? it can rubber-
1079             stamp a real \"
1080             \"regression; record the deliberate reason and review the visible
1081             diff ($6).\")
1082             targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,
1083             fixtures_dir=fixtures_dir)
1084             print(f\"[golden-fixtures] refreshed snapshot for: {' '.join(targets)}\")
1085             f\" (reason: {args.reason})\"
1086             return 0
1087
1088     if args.freeze_real:
1089         missing = [n for n, v in ((\"--trajectory\", args.trajectory),
1090             (\"--labels\", args.labels),
1091             (\"--fps\", args.fps),
1092             (\"--frame-width\", args.frame_width)) if v is None]
1093         if missing:
1094             ap.error(\"--freeze-real requires \" + \" \".join(missing))
1095         try:
1096             expected = freeze_real_fixture(
1097                 args.label, args.trajectory, args.labels,
1098                 license=args.license_ if args.license_ is not None else \"\",
1099                 minors_free=args.minors_free, owned=args.owned,
1100                 fps=args.fps, frame_width=args.frame_width,
1101                 fixtures_dir=fixtures_dir, slug=args.slug,
1102                 time_origin_reset=args.time_origin_reset, source_note=args.source_note)
1103             except RefusalError as e:
1104                 # The code-enforced provenance gate ($4c / $7 ?): refuse loudly, exit non-
1105                 zero.
1106                 print(f\"[golden-fixtures] REFUSED (--freeze-real provenance gate):\\n{e}\",
1107                 file=sys.stderr)
1108                 return 2
1109             print(f\"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}'
1110             \"
1111             f\"(kind=cleared_real) -> {fixtures_dir / expected['slug']}\")
1112             print(\" $6: behaviour-locking, NOT correctness; non-attributable only relative
1113             to the \"
1114             \"UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
1115             (P2).\")
1116             return 0
1117
1118     # --check
1119     return _cmd_check(fixtures_dir)
1120
1121
1122 if __name__ == \"__main__\":
1123     raise SystemExit(main())
1124

```

7. assistant (2026-06-18T10:21:12.622Z)

Now let me look at the existing test files to understand what's already there and the test split structure.

8. user (2026-06-18T10:21:14.215Z)

test_fusion_golden_telemetry.py
test_golden_fixtures.py
test_golden_real_fixtures.py

9. user (2026-06-18T10:21:17.025Z)

```
1      """Golden Regression Fixtures (P1 + M2) ? de-attribution / minimization + the refusal
gate.
2
3      These tests exercise ``golden_fixtures.freeze_real_fixture`` (the P1 tooling that turns
an OWNED,
4      minors-free real trajectory + golden labels into a committable, de-attributed Tier-0
fixture) and
5      its M2 ``--license`` / ``--minors-free`` / ``--owned`` refusal gate.
6
7      Everything runs on SYNTHETIC input written into ``tmp_path`` ? NO real data, NO network,
NO GPU.
8      We reuse M1's synthetic generators (``builtin_specs`` / ``make_synthetic_trajectory``
and the
9      ``write_trajectory_csv`` / ``write_labels_csv`` writers) to manufacture the "source"
CSVs, then
10     prove:
11         * the refusal gate blocks every unsafe combination (M2);
12         * a successful freeze is de-attributed (random ``fx`` slug, no id/name/path, no
FORBIDDEN_KEYS);
13         * the time-origin reset is metric-INVARIANT (score identical with/without; only
absolute
14         ``control_segments`` shift toward 0);
15         * the freeze round-trips (``replay_fixture`` + ``compare_expected`` == []).
16
17     $6 honest-limits caveat: a green suite proves only that the deterministic post-
trajectory transforms
18     are unchanged for the frozen cases ? characterization is behaviour-locking, NOT
correctness.
19     """
20     import pytest
21
22     from backend.eval import golden_fixtures as gf
23
24
25     # -----
26     # Helpers ? manufacture a SYNTHETIC "real" source (trajectory + labels CSV) in tmp_path.
27     # -----
28
29
30     def _write_source(tmp_path, *, frame_offset: int = 0):
31         """Write a synthetic source trajectory+labels CSV pair into ``tmp_path``.
32
33         Reuses M1's first built-in scenario (a clean single rally), optionally shifted by
34         ``frame_offset`` frames (and the matching label seconds) to model an arbitrary
absolute
35         capture timeline. Returns ``(traj_path, labels_path, fps, frame_width, gts)``.
36         """
37         spec = gf.builtin_specs()[0] # clean_single_rally
38         fps, fw = spec.fps, spec.frame_width
39         rows = gf.make_synthetic_trajectory(spec)
40         if frame_offset:
41             rows = [(f + frame_offset, v, x, y) for (f, v, x, y) in rows]
42         gts = list(spec.gts)
43         if frame_offset:
44             off_sec = frame_offset / fps
45             gts = [(s + off_sec, e + off_sec) for (s, e) in gts]
46
47         traj = tmp_path / "src_trajectory.csv"
48         labels = tmp_path / "src_labels.csv"
```

```

49         gf.write_trajectory_csv(rows, traj)
50         gf.write_labels_csv(gts, labels)
51         return traj, labels, fps, fw, gts
52
53
54     # -----
55     # M2 ? refusal gate
56     # -----
57
58
59     @pytest.mark.parametrize(
60         "label, kwargs, why",
61         [
62             ("scenario", {"minors_free": False, "owned": True, "license": "owned"},
63 "minors_free=False"),
64             ("scenario", {"minors_free": True, "owned": False, "license": "owned"},
65 "owned=False"),
66             ("scenario", {"minors_free": True, "owned": True, "license": ""}, "license=''),
67             ("scenario", {"minors_free": True, "owned": True, "license": "   "},
68 "license=blank"),
69             ("scenario", {"minors_free": True, "owned": True, "license": None},
70 "license=None"),
71             # red-team #1: a free-text identifier label (name / event / spaces /
72 punctuation) is refused.
73             ("Lin Dan vs Lee, India Open 2024",
74 {"minors_free": True, "owned": True, "license": "owned"}, "label=player-name
75 free text"),
76             ("mens singles final",
77 {"minors_free": True, "owned": True, "license": "owned"}, "label=spaces"),
78             ("Final_2024", {"minors_free": True, "owned": True, "license": "owned"},
79 "label=uppercase"),
80             ("", {"minors_free": True, "owned": True, "license": "owned"}, "label=empty"),
81         ],
82     )
83
84     def test_refusal_gate_blocks_unsafe(tmp_path, label, kwargs, why):
85         """freeze_real_fixture refuses (ValueError) unless minors_free AND owned AND a non-
86 blank license
87 AND a STRICT-slug label (no free-text identifier ? red-team #1).
88
89 The refusal is the code-enforced provenance gate ($4c / $7 ?): the unsafe public-
90 fixture path
91 must be impossible to reach silently. A refused freeze leaves NO partial fixture dir
92 (red-team
93 #3b: write-to-temp-then-promote).
94 """
95         traj, labels, fps, fw, _ = _write_source(tmp_path)
96         with pytest.raises(ValueError) as ei:
97             gf.freeze_real_fixture(
98                 label, traj, labels,
99                 fps=fps, frame_width=fw, fixtures_dir=tmp_path / "fixtures", **kwargs)
100         # RefusalError is a ValueError subclass; the message must name a reason.
101         assert isinstance(ei.value, gf.RefusalError), why
102         assert "REFUSED" in str(ei.value), why
103         # Nothing was written for a refused freeze (no fixture dir, partial or otherwise).
104         assert not (tmp_path / "fixtures").exists() or not any((tmp_path /
105 "fixtures").iterdir()), (
106             f"{why}: a refused freeze must not write any fixture")
107
108     # -----
109     # P1 ? de-attribution
110     # -----
111
112     def test_real_fixture_is_deattributed(tmp_path):

```

```

103         """A successful safe freeze is de-attributed: kind=cleared_real, fx_-prefixed
surrogate slug,
104         no id/name/filename/source-path key, and NO FORBIDDEN_KEYS in expected.json."""
105         fixtures = tmp_path / "fixtures"
106         traj, labels, fps, fw, _ = _write_source(tmp_path)
107         expected = gf.freeze_real_fixture(
108             "clean_rally", traj, labels,
109             license="owned", minors_free=True, owned=True,
110             fps=fps, frame_width=fw, fixtures_dir=fixtures)
111
112         slug = expected["slug"]
113         assert slug.startswith("fx_"), f"surrogate slug must be opaque fx_<hex>, got
{slug!r}"
114
115         prov = gf._read_json(fixtures / slug / "provenance.json")
116         assert prov["kind"] == "cleared_real"
117         assert prov["minors_free"] is True and prov["owned"] is True
118         assert prov["license"] == "owned"
119         assert prov["slug"] == slug
120         assert prov["label"] == "clean_rally" # strict-slug label is committed verbatim
121         # No attribution keys / no source filename anywhere in the provenance. `source_note`
is now
122         # DROPPED entirely (red-team #1b): operator free-text never reaches the committed
bytes.
123         for banned in ("video_id", "name", "filename", "path", "match", "capture_date",
"source_note"):
124             assert banned not in prov, f"provenance must not carry an attribution key
{banned!r}"
125             prov_text = (fixtures / slug /
"provenance.json").read_text(encoding="utf-8").lower()
126             for tok in ("src_trajectory", "src_labels", str(traj).lower(), str(labels).lower()):
127                 assert tok not in prov_text, f"source token {tok!r} leaked into provenance"
128
129             # expected.json carries no biometric/identity/non-shuttle stream.
130             exp_text = (fixtures / slug / "expected.json").read_text(encoding="utf-8").lower()
131             for forbidden in gf.FORBIDDEN_KEYS:
132                 assert forbidden not in exp_text, f"FORBIDDEN_KEYS leak {forbidden!r} in
expected.json"
133
134             # The fixture is registered in the manifest with kind=cleared_real.
135             manifest = gf.load_manifest(fixtures)
136             rows = [e for e in manifest if e["slug"] == slug]
137             assert len(rows) == 1 and rows[0]["kind"] == "cleared_real"
138
139
140     def test_supplied_slug_is_honored(tmp_path):
141         """An explicitly supplied slug is used verbatim (the surrogate map is the caller's
concern)."""
142         fixtures = tmp_path / "fixtures"
143         traj, labels, fps, fw, _ = _write_source(tmp_path)
144         expected = gf.freeze_real_fixture(
145             "scenario", traj, labels, slug="fx_custom00",
146             license="owned", minors_free=True, owned=True,
147             fps=fps, frame_width=fw, fixtures_dir=fixtures)
148         assert expected["slug"] == "fx_custom00"
149         assert (fixtures / "fx_custom00" / "expected.json").is_file()
150
151
152     def test_freeze_real_does_not_clobber_existing_manifest(tmp_path):
153         """Freezing a second real fixture must APPEND to the manifest, not wipe the
first."""
154         fixtures = tmp_path / "fixtures"
155         traj, labels, fps, fw, _ = _write_source(tmp_path)
156         e1 = gf.freeze_real_fixture("a", traj, labels, slug="fx_aaaa0001",
157                                     license="owned", minors_free=True, owned=True,

```

```

158             fps=fps, frame_width=fw, fixtures_dir=fixtures)
159     e2 = gf.freeze_real_fixture("b", traj, labels, slug="fx_bbbb0002",
160                                license="owned", minors_free=True, owned=True,
161                                fps=fps, frame_width=fw, fixtures_dir=fixtures)
162     slugs = {e["slug"] for e in gf.load_manifest(fixtures)}
163     assert {e1["slug"], e2["slug"]} <= slugs
164
165
166     def test_positive_privacy_assertion_catches_leak_in_source_note(tmp_path):
167         """source_note is ACCEPTED-BUT-NOT-PERSISTED (red-team #1b), yet still scanned
168         defensively ? a
169         path/name in it trips the positive scan and refuses (and leaves no partial dir, red-
170         team #3b)."""
171         fixtures = tmp_path / "fixtures"
172         traj, labels, fps, fw, _ = _write_source(tmp_path)
173         with pytest.raises(gf.RefusalError):
174             gf.freeze_real_fixture(
175                 "scenario", traj, labels, slug="fx_leak0001",
176                 license="owned", minors_free=True, owned=True,
177                 fps=fps, frame_width=fw, fixtures_dir=fixtures,
178                 source_note=str(traj)) # leaking the source path ? must be refused
179         # The refused freeze left no fixture dir behind (write-to-temp-then-promote).
180         assert not (fixtures / "fx_leak0001").exists(), "refused freeze must not leave a
181         partial dir"
182
183     def test_source_note_safe_is_not_persisted(tmp_path):
184         """A SAFE source_note (no path/name) is accepted but NEVER written to the committed
185         provenance
186         (red-team #1b: operator free-text is an attribution back-channel, dropped from
187         committed bytes)."""
188         fixtures = tmp_path / "fixtures"
189         traj, labels, fps, fw, _ = _write_source(tmp_path)
190         expected = gf.freeze_real_fixture(
191             "scenario", traj, labels, slug="fx_note0001",
192             license="owned", minors_free=True, owned=True,
193             fps=fps, frame_width=fw, fixtures_dir=fixtures,
194             source_note="benign generic note")
195         prov = gf._read_json(fixtures / expected["slug"] / "provenance.json")
196         assert "source_note" not in prov, "source_note must NOT be persisted into
197         provenance"
198         prov_text = (fixtures / expected["slug"] /
199         "provenance.json").read_text(encoding="utf-8")
200         assert "benign generic note" not in prov_text, "source_note text leaked into
201         committed bytes"
202
203     def test_freeze_real_label_slug_committed(tmp_path):
204         """A valid strict-slug label is committed verbatim; a free-text label is refused
205         (red-team #1a)."""
206         fixtures = tmp_path / "fixtures"
207         traj, labels, fps, fw, _ = _write_source(tmp_path)
208         ok = gf.freeze_real_fixture(
209             "mens-singles_final-01", traj, labels, slug="fx_lab00001",
210             license="owned", minors_free=True, owned=True,
211             fps=fps, frame_width=fw, fixtures_dir=fixtures)
212         prov = gf._read_json(fixtures / ok["slug"] / "provenance.json")
213         assert prov["label"] == "mens-singles_final-01"
214         with pytest.raises(gf.RefusalError):
215             gf.freeze_real_fixture(
216                 "Lin Dan vs Lee 2024", traj, labels, slug="fx_lab00002",
217                 license="owned", minors_free=True, owned=True,
218                 fps=fps, frame_width=fw, fixtures_dir=fixtures)

```

```

214     def test_freeze_real_negative_reset_label_refused(tmp_path):
215         """A label timeline that predates the earliest trajectory frame ? negative reset
start ? refused
216         BEFORE any write, leaving no partial fixture dir (red-team #3a)."""
217         fixtures = tmp_path / "fixtures"
218         spec = gf.builtin_specs()[0]
219         fps, fw = spec.fps, spec.frame_width
220         # Trajectory starts at frame 300 (t0 ? 10s) but a label starts at t=0 ? reset start
< 0.
221         rows = [(f + 300, v, x, y) for (f, v, x, y) in gf.make_synthetic_trajectory(spec)]
222         traj = tmp_path / "src_trajectory.csv"
223         labels = tmp_path / "src_labels.csv"
224         gf.write_trajectory_csv(rows, traj)
225         gf.write_labels_csv([(0.0, 4.0)], labels)
226         with pytest.raises(gf.RefusalError) as ei:
227             gf.freeze_real_fixture(
228                 "scenario", traj, labels, slug="fx_neg00001",
229                 license="owned", minors_free=True, owned=True,
230                 fps=fps, frame_width=fw, fixtures_dir=fixtures, time_origin_reset=True)
231         assert "predates the trajectory" in str(ei.value)
232         assert not (fixtures / "fx_neg00001").exists(), "refused freeze must not leave a
partial dir"
233
234
235     def test_scan_forbidden_catches_label_csv_source_stem(tmp_path):
236         """red-team #4: a real source named ``label.csv`` (stem ``label`` ? ``labels.csv``)
must still
237         be caught ? the source-path scan uses EXACT schema-literal membership, never
substring-skip."""
238         # A source-path token "label" is NOT an exact schema literal, so it must trip the
scan when it
239         # appears in the written text. We feed it via a source path whose stem is "label".
240         text = '{"slug": "fx_x", "note": "label appears here"}'
241         with pytest.raises(gf.RefusalError) as ei:
242             gf._scan_forbidden(text, where="x/expected.json",
243                               source_paths=["/some/dir/label.csv"])
244         assert "source-path token" in str(ei.value) and "label" in str(ei.value)
245
246
247     def test_scan_forbidden_allows_exact_schema_literal(tmp_path):
248         """A source path that EXACTLY equals a schema literal (e.g. ``trajectory.csv``) is
not a leak ?
249         those bytes are written regardless, so the scan must not false-positive on them."""
250         text = '{"slug": "fx_x", "paths": {"trajectory": "trajectory.csv", "labels":
"labels.csv"}}'
251         # Source basename == schema literal "trajectory.csv": must NOT raise.
252         gf._scan_forbidden(text, where="x/expected.json",
source_paths=["/some/dir/trajectory.csv"])
253
254
255     # -----
256     # P1 (d) ? time-origin reset is metric-invariant
257     # -----
258
259
260     def _raw_score_of_frozen(fdir):
261         """Re-score a frozen fixture from its committed CSVs through the SHIPPED stack
WITHOUT the
262         snapshot's final 6dp rounding ? so the test sees the genuine metric, not the
serialization grid.
263
264         (``replay_fixture`` rounds every float to 6dp for the committed snapshot; on a sub-
second
265         time-origin reset two boundary errors can land on adjacent 6dp grid points ? a 1e-6
SERIALIZED

```

```

266         gap that is the grid, not a metric divergence. The raw score below exposes the true
delta.)
267         ""
268         from backend.eval import distill_local
269         from backend.eval.experiment import CONTROL, VideoCandidates, predict
270         from backend.eval.gt_loader import load_gt_intervals
271         from backend.eval.metrics import score
272         intervals, feature_rows = distill_local.build_candidates(
273             str(fdir / "trajectory.csv"), fps=30.0, frame_width=1280.0, proposal=(0.02, 2.0,
2.0))
274         feats = {n: [r[i] for r in feature_rows] for i, n in enumerate(gf.SHUTTLE_FEATURES)}
275         gts = load_gt_intervals(str(fdir / "labels.csv"), strict=True)
276         preds = predict(CONTROL, VideoCandidates("x", intervals, feats, gts))
277         return score(preds, gts, 0.5)
278
279
280     # Fractional-second offsets (7, 11, 13 frames @30fps) plus whole-second (9000 = 300.0s)
and a few
281     # more sub-second offsets ? every one must be metric-invariant within _FLOAT_TOL (red-
team #2).
282     @pytest.mark.parametrize("offset_frames", [1, 7, 11, 13, 17, 9000])
283     def test_time_origin_reset_is_metric_invariant(tmp_path, offset_frames):
284         ""A source frozen WITH reset scores within ``_FLOAT_TOL`` of the SAME source frozen
WITHOUT
285         reset, for ANY fractional-second offset ? and its absolute control_segments shift
toward 0
286         (severed timeline). Before fix #2 the boundary error drifted to 1.000000000001e-06
(just past
287         _FLOAT_TOL) on a sub-second offset; rounding t0_sec + each label endpoint to 6dp
before the
288         shift keeps the genuine (raw) delta at the float-error floor (~3.3e-7).""
289         traj, labels, fps, fw, _ = _write_source(tmp_path, frame_offset=offset_frames)
290
291         with_reset = gf.freeze_real_fixture(
292             "scenario", traj, labels, slug="fx_reset_on",
293             license="owned", minors_free=True, owned=True,
294             fps=fps, frame_width=fw, fixtures_dir=tmp_path / "with",
295             time_origin_reset=True)
296         without_reset = gf.freeze_real_fixture(
297             "scenario", traj, labels, slug="fx_reset_off",
298             license="owned", minors_free=True, owned=True,
299             fps=fps, frame_width=fw, fixtures_dir=tmp_path / "without",
300             time_origin_reset=False)
301
302         # (1) Structural fields (counts/TP/FP/FN) are EXACTLY equal regardless of offset.
303         for k in ("num_pred", "num_gt", "true_positives", "false_positives",
"false_negatives"):
304             assert with_reset["score"][k] == without_reset["score"][k], f"{k} changed under
reset"
305
306         # (2) Every float score field is metric-invariant within _FLOAT_TOL on the RAW (un-
rounded)
307         #     score ? the genuine metric, not the 6dp serialization grid (see
_raw_score_of_frozen).
308         raw_on = _raw_score_of_frozen(tmp_path / "with" / "fx_reset_on")
309         raw_off = _raw_score_of_frozen(tmp_path / "without" / "fx_reset_off")
310         for f in ("precision", "recall", "f1", "mean_iou", "mean_start_error",
"mean_end_error"):
311             delta = abs(getattr(raw_on, f) - getattr(raw_off, f))
312             assert delta <= gf._FLOAT_TOL, (
313                 f"offset={offset_frames}: {f} not metric-invariant under time-origin reset:
"
314                 f"|?|={delta!r} > _FLOAT_TOL={gf._FLOAT_TOL!r}")
315
316         # (3) A whole-SECOND offset is additionally BIT-identical on the serialized snapshot

```

```

(the 6dp
317     # grid lands cleanly), proving the reset itself introduces no real divergence.
318     if offset_frames % int(fps) == 0:
319         assert with_reset["score"] == without_reset["score"], (
320             "whole-second offset must be bit-identical on the serialized snapshot")
321
322     # (4) Absolute timeline differs ? the reset pulls control_segments toward 0.
323     seg_on = with_reset["control_segments"]
324     seg_off = without_reset["control_segments"]
325     assert len(seg_on) == len(seg_off) and len(seg_on) >= 1
326     assert seg_on[0][0] < seg_off[0][0], "reset must shift the first segment start
toward 0"
327     shift = seg_off[0][0] - seg_on[0][0]
328     assert abs(shift - offset_frames / fps) < 1e-3, (
329         f"segment shift {shift} should equal the frame offset in seconds {offset_frames
/ fps}")
330     # Reset trajectory starts at frame 0.
331     first_frame = int((tmp_path / "with" / "fx_reset_on" / "trajectory.csv")
332         .read_text(encoding="utf-8").splitlines()[1].split(",")[0])
333     assert first_frame == 0, "with reset, the trajectory must start at Frame 0"
334
335
336     # -----
337     # P1 (e/f) ? round-trip / correct-by-construction
338     # -----
339
340
341     def test_freeze_real_round_trips_check(tmp_path):
342         """After freeze_real_fixture, replay_fixture + compare_expected vs the written
expected.json
343         returns [] ? idempotent / correct-by-construction (the same path M1's --check
uses)."""
344         fixtures = tmp_path / "fixtures"
345         traj, labels, fps, fw, _ = _write_source(tmp_path)
346         expected = gf.freeze_real_fixture(
347             "scenario", traj, labels, slug="fx_rt000001",
348             license="owned", minors_free=True, owned=True,
349             fps=fps, frame_width=fw, fixtures_dir=fixtures)
350
351         fdir = fixtures / expected["slug"]
352         committed = gf._read_json(fdir / "expected.json")
353         recomputed = gf.replay_fixture(fdir)
354         assert gf.compare_expected(recomputed, committed) == [], (
355             "a just-frozen real fixture must replay-compare clean (correct-by-
construction)")
356
357
358     def test_freeze_real_uses_strict_label_loader(tmp_path):
359         """The labels are read through the STRICT gt_loader ? a malformed label row fails
loudly,
360         never silently shrinks the GT set."""
361         fixtures = tmp_path / "fixtures"
362         traj, labels, fps, fw, _ = _write_source(tmp_path)
363         # Corrupt one label row (end <= start) ? strict loader must raise.
364         labels.write_text("start,end\n5.0,1.0\n", encoding="utf-8", newline="\n")
365         with pytest.raises(ValueError):
366             gf.freeze_real_fixture(
367                 "scenario", traj, labels, slug="fx_strict01",
368                 license="owned", minors_free=True, owned=True,
369                 fps=fps, frame_width=fw, fixtures_dir=fixtures)
370
371
372     def test_freeze_real_refuses_to_clobber_existing_dir(tmp_path):
373         """Freezing into a slug dir that already exists is refused (no silent overwrite)."""
374         fixtures = tmp_path / "fixtures"

```

```

375     traj, labels, fps, fw, _ = _write_source(tmp_path)
376     gf.freeze_real_fixture("a", traj, labels, slug="fx_dup00001",
377                           license="owned", minors_free=True, owned=True,
378                           fps=fps, frame_width=fw, fixtures_dir=fixtures)
379     with pytest.raises(ValueError):
380         gf.freeze_real_fixture("b", traj, labels, slug="fx_dup00001",
381                               license="owned", minors_free=True, owned=True,
382                               fps=fps, frame_width=fw, fixtures_dir=fixtures)
383
384
385     # -----
386     # FAILURE semantics ? the gates must FAIL loudly, not silently pass (red-team #10, #5,
387     # -----
388
389
390     def _good_snapshot(tmp_path):
391         """Freeze a clean real fixture and return its committed expected.json snapshot
392         dict."""
393         fixtures = tmp_path / "fixtures"
394         traj, labels, fps, fw, _ = _write_source(tmp_path)
395         return gf.freeze_real_fixture(
396             "scenario", traj, labels, slug="fx_snap00001",
397             license="owned", minors_free=True, owned=True,
398             fps=fps, frame_width=fw, fixtures_dir=fixtures)
399
400     def test_compare_expected_detects_changed_float(tmp_path):
401         """A float field changed by > 1e-6 is a mismatch (characterization catches behaviour
402         drift)."""
403         import copy
404         snap = _good_snapshot(tmp_path)
405         other = copy.deepcopy(snap)
406         other["score"]["f1"] = snap["score"]["f1"] - 1e-3
407         assert gf.compare_expected(other, snap), "a >1e-6 float change must be a non-empty
408         mismatch list"
409
410
411     def test_compare_expected_detects_changed_int(tmp_path):
412         """A changed int/structural field is a mismatch."""
413         import copy
414         snap = _good_snapshot(tmp_path)
415         other = copy.deepcopy(snap)
416         other["score"]["true_positives"] = snap["score"]["true_positives"] + 1
417         assert gf.compare_expected(other, snap)
418
419
420     def test_compare_expected_detects_candidate_reorder(tmp_path):
421         """Reordering candidates is a mismatch (order is structural ? first-vs-second window
422         matters)."""
423         import copy
424         # Use a 2-candidate scenario so a reorder is observable.
425         spec = gf.builtin_specs()[1] # multi_rally_with_deadtime
426         rows = gf.make_synthetic_trajectory(spec)
427         traj = tmp_path / "src_trajectory.csv"
428         labels = tmp_path / "src_labels.csv"
429         gf.write_trajectory_csv(rows, traj)
430         gf.write_labels_csv(spec.gts, labels)
431         snap = gf.freeze_real_fixture(
432             "scenario", traj, labels, slug="fx_reord001",
433             license="owned", minors_free=True, owned=True,
434             fps=spec.fps, frame_width=spec.frame_width, fixtures_dir=tmp_path / "fixtures")
435         assert len(snap["candidates"]) >= 2
436         other = copy.deepcopy(snap)
437         other["candidates"] = list(reversed(other["candidates"]))

```

```

435         assert gf.compare_expected(other, snap)
436
437
438     def test_compare_expected_detects_extra_top_level_key(tmp_path):
439         """red-team #5: an EXTRA top-level key on one side is a mismatch (key-set
equality)."""
440         import copy
441         snap = _good_snapshot(tmp_path)
442         other = copy.deepcopy(snap)
443         other["unexpected_field"] = 1
444         mism = gf.compare_expected(other, snap)
445         assert mism and any("top-level" in m for m in mism), mism
446
447
448     def test_compare_expected_detects_extra_score_key(tmp_path):
449         """red-team #5: an EXTRA key inside the score block is a mismatch (key-set
equality)."""
450         import copy
451         snap = _good_snapshot(tmp_path)
452         other = copy.deepcopy(snap)
453         other["score"]["bogus_metric"] = 0.0
454         mism = gf.compare_expected(other, snap)
455         assert mism and any(m.startswith("score:") for m in mism), mism
456
457
458     def test_scan_forbidden_raises_on_forbidden_key():
459         """red-team #10: the FORBIDDEN_KEYS branch raises (a person/audio/pose stream in the
text)."""
460         with pytest.raises(gf.RefusalError) as ei:
461             gf._scan_forbidden({'person': [1,2,3]}', where="x", source_paths=[])
462         assert "forbidden key" in str(ei.value)
463
464
465     def test_scan_forbidden_raises_on_md5():
466         """red-team #10: the MD5 branch raises (a 32-hex video_id shape is an attribution
back-channel)."""
467         md5 = "0123456789abcdef0123456789abcdef"
468         with pytest.raises(gf.RefusalError) as ei:
469             gf._scan_forbidden({'slug': "%s"} % md5, where="x", source_paths=[])
470         assert "MD5" in str(ei.value)
471
472
473     def test_refresh_snapshot_no_target_raises(tmp_path):
474         """red-team #10: refresh_snapshot with neither slug nor all raises ValueError."""
475         with pytest.raises(ValueError):
476             gf.refresh_snapshot(fixtures_dir=tmp_path)
477
478
479     def test_refresh_snapshot_cli_requires_reason(capsys):
480         """red-team #6: --refresh-snapshot --all WITHOUT --reason is a CLI error (argparse
exits 2)."""
481         with pytest.raises(SystemExit) as ei:
482             gf.main(["--refresh-snapshot", "--all"])
483         assert ei.value.code == 2 # argparse ap.error exit code
484         err = capsys.readouterr().err
485         assert "--reason" in err
486
487
488     def test_load_fixture_guards_provenance_schema_independently(tmp_path):
489         """red-team #7: a provenance schema_version > CURRENT (with expected sv in-range)
still raises ?
490         the two blocks are guarded INDEPENDENTLY, not masked by each other."""
491         fixtures = tmp_path / "fixtures"
492         traj, labels, fps, fw, _ = _write_source(tmp_path)
493         expected = gf.freeze_real_fixture(

```

```

494         "scenario", traj, labels, slug="fx_sv000001",
495         license="owned", minors_free=True, owned=True,
496         fps=fps, frame_width=fw, fixtures_dir=fixtures)
497     slug = expected["slug"]
498     # expected.json stays at the current schema; bump ONLY provenance to a future
version.
499     prov = gf._read_json(fixtures / slug / "provenance.json")
500     prov["schema_version"] = 99
501     gf._write_json(fixtures / slug / "provenance.json", prov)
502     with pytest.raises(ValueError) as ei:
503         gf.load_fixture(slug, fixtures_dir=fixtures)
504     assert "provenance schema_version 99" in str(ei.value)
505

```

10. assistant (2026-06-18T10:21:19.403Z)

Now let me look at the existing `test\_golden\_fixtures.py` to understand what the shared-primitives test file looks like, and confirm which names need to move. Let me identify exactly what `freeze\_real\_fixture` depends on.

11. user (2026-06-18T10:21:21.328Z)

```

1     """Golden Regression Fixtures (M1) ? characterization tests over the committed synthetic
Tier-0.
2
3     Parametrized over ``load_manifest()`` so every committed fixture is exercised. The gate
is
4     **parse-compare** (never byte-compare), **CONTROL-only** (pure stdlib, no numpy on the
replay
5     path), and asserts the Tier-0 privacy invariant positively (no
person/audio/pose/gait/face keys).
6
7     $6 honest-limits caveat: green here proves only that the deterministic post-trajectory
transforms
8     are unchanged for the frozen synthetic cases ? characterization is behaviour-locking,
NOT
9     correctness, and synthetic-tier numbers measure plumbing, not field quality.
10    """
11    import json
12
13    import pytest
14
15    from backend.eval import golden_fixtures as gf
16
17    # Slugs discovered from the committed manifest ? the suite grows by accretion
automatically.
18    MANIFEST = gf.load_manifest()
19    SLUGS = [entry["slug"] for entry in MANIFEST]
20
21
22    def test_manifest_is_non_empty():
23        """A committed manifest with at least one fixture must exist (else nothing is
gated)."""
24        assert SLUGS, "no fixtures in eval_baselines/fixtures/manifest.json ? run --freeze-
synthetic"
25
26
27    @pytest.mark.parametrize("slug", SLUGS)
28    def test_replay_matches_committed_expected(slug):
29        """Replay the fixture and parse-compare against its committed expected.json.
30
31        Characterization = behaviour-locking, NOT correctness ($6): a non-empty mismatch
list means
32        the deterministic post-trajectory behaviour CHANGED for this frozen case, not that
it is wrong.

```

```

33         """
34         committed = gf.load_fixture(slug)["expected"]
35         recomputed = gf.replay_fixture(slug)
36         mismatches = gf.compare_expected(recomputed, committed)
37         assert mismatches == [], (
38             f"{slug}: behaviour changed vs frozen snapshot (characterization, not
correctness ? §6):\n"
39             + "\n".join(f" - {m}" for m in mismatches)
40         )
41
42
43     @pytest.mark.parametrize("slug", SLUGS)
44     def test_schema_version_le_current(slug):
45         """Every committed fixture's schema_version must be <= CURRENT_SCHEMA (forward-
compatible)."""
46         fx = gf.load_fixture(slug)
47         assert int(fx["expected"]["schema_version"]) <= gf.CURRENT_SCHEMA
48         assert int(fx["provenance"]["schema_version"]) <= gf.CURRENT_SCHEMA
49
50
51     @pytest.mark.parametrize("slug", SLUGS)
52     def test_no_person_or_audio_keys(slug):
53         """Positive privacy assertion (D3 / §4c): NO biometric/identity/non-shuttle key
appears
54         anywhere in the committed expected.json ? not trusting a silent 0.0 default."""
55         fdir = gf.FIXTURES_DIR / slug
56         raw = (fdir / "expected.json").read_text(encoding="utf-8").lower()
57         for forbidden in gf.FORBIDDEN_KEYS:
58             assert forbidden not in raw, (
59                 f"{slug}: forbidden key substring {forbidden!r} present in expected.json
(Tier-0 leak)")
60

```

12. assistant (2026-06-18T10:21:29.080Z)

Now I have a clear picture. Let me analyze the dependency graph carefully to plan the split. Let me identify exactly which symbols ``freeze_real_fixture``, ``RefusalError``, and ``_scan_forbidden`` need.

The real-fixture-only functions are:

- ``RefusalError`` (class)
- ``_new_surrogate_slug``
- ``_SCHEMA_FILENAME_LITERALS``
- ``_scan_forbidden``
- ``_reset_trajectory_rows``
- ``_reset_labels``
- ``freeze_real_fixture``

These depend on shared primitives from ``golden_fixtures``:

- Constants: ``CURRENT_SCHEMA``, ``FIXTURES_DIR``, ``PINNED_VT``, ``PINNED_MERGE_GAP``, ``PINNED_MIN_WINDOW``, ``PINNED_MAX_WINDOW``, ``FORBIDDEN_KEYS``, ``_MD5_RE``, ``_LABEL_SLUG_RE``, ``_FLOAT_ROUND_DP``
- Functions: ``_round``, ``write_trajectory_csv``, ``write_labels_csv``, ``_write_json``, ``replay_fixture``, ``_append_manifest_entry``, and uses of ``gt_hash``, ``load_gt_intervals``, ``TrackNetRunner``

Let me check the regression module and any other importers of these names to ensure the re-export keeps all import paths valid.

13. user (2026-06-18T10:21:30.571Z)

Found 4 files

```

tests\test_golden_real_fixtures.py
docs\GOLDEN_REGRESSION_FIXTURES.md
docs\CODE_MAP.md

```

backend\eval\golden_fixtures.py

14. user (2026-06-18T10:21:33.614Z)

Found 6 files
tests\test_golden_real_fixtures.py
tests\test_golden_fixtures.py
eval_baselines\fixtures\README.md
docs\GOLDEN_REGRESSION_FIXTURES.md
docs\CODE_MAP.md
.github\workflows\ci.yml

15. assistant (2026-06-18T10:21:35.784Z)

Let me check how the CI workflow and regression module reference these, to confirm import paths stay valid.

16. user (2026-06-18T10:21:37.929Z)

```
.github/workflows/ci.yml:99: # Cross-OS determinism guard for the committed golden regression
fixtures
.github/workflows/ci.yml:105: golden-crossos:
.github/workflows/ci.yml:124:     python -m backend.eval.golden_fixtures --check
.github/workflows/ci.yml:125:     python -m pytest tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py -q
eval_baselines/fixtures/README.md:19:`manifest.json` lists every fixture;
`backend/eval/golden_fixtures.py` is the shared replay
eval_baselines/fixtures/README.md:51:python -m backend.eval.golden_fixtures --freeze-synthetic
eval_baselines/fixtures/README.md:54:python -m backend.eval.golden_fixtures --check
eval_baselines/fixtures/README.md:58:python -m backend.eval.golden_fixtures --refresh-snapshot
--all --reason "<why>"
eval_baselines/fixtures/README.md:59:python -m backend.eval.golden_fixtures --refresh-snapshot
--slug clean_single_rally --reason "<why>"
```

17. assistant (2026-06-18T10:21:45.839Z)

All references go through `golden\_fixtures` (the module path stays stable) plus the test files. The re-export plan keeps `backend.eval.golden\_fixtures.freeze\_real\_fixture` / `RefusalError` / `\_scan\_forbidden` valid. The tests use `gf.freeze\_real\_fixture`, `gf.RefusalError`, `gf.\_scan\_forbidden`, `gf.\_new\_surrogate\_slug`? Let me check ? the tests reference `gf.\_read\_json`, `gf.\_write\_json`, `gf.replay\_fixture`, `gf.load\_manifest`, `gf.compare\_expected`, `gf.SHUTTLE\_FEATURES`, `gf.\_FLOAT\_TOL`, `gf.FORBIDDEN\_KEYS`, `gf.builtin\_specs`, `gf.make\_synthetic\_trajectory`, `gf.write\_trajectory\_csv`, `gf.write\_labels\_csv`, `gf.refresh\_snapshot`, `gf.main`, `gf.load\_fixture`. All of those that the real-fixture tests use stay in `golden\_fixtures` (shared) except `freeze\_real\_fixture`, `RefusalError`, `\_scan\_forbidden` which must be re-exported.

First, let me run the existing tests on the UNCHANGED code to confirm they pass (characterization baseline).

18. user (2026-06-18T10:21:50.927Z)

```
..... [100%]
54 passed in 2.44s
```

19. assistant (2026-06-18T10:21:58.473Z)

54 passed on unchanged code. Now let me write a small characterization test that pins the identity (same-object via re-export) behavior. This test must FAIL on unchanged code if I assert `golden\_real\_fixtures` exists ? but the new module doesn't exist yet. The task says to add a test asserting `golden\_fixtures.freeze\_real\_fixture` is `golden_real_fixtures.freeze_real_fixture`. That test can only pass AFTER the split.

Per the discipline, characterization tests pin CURRENT behavior of the code I touch. The current

behavior I'm pinning is that these three names are accessible via `golden\_fixtures` and produce specific results. Let me write a characterization test file that pins the current behavior (importable + identity within the module + functional smoke) ? and I'll write the identity-across-modules assertion to be added now but it pins the post-split contract. Let me structure it so the pre-refactor run validates the parts that exist today, and I'll confirm.

Let me first add characterization assertions that pin today's behavior (the names exist on `golden\_fixtures` and are callable), then add the cross-module identity test which I'll verify after the split.

20. assistant (2026-06-18T10:22:17.746Z)

```
"""Characterization tests pinning the ``golden_fixtures`` ? ``golden_real_fixtures`` module split.
```

This file locks the import-surface contract that the real-fixture-subsystem split must preserve: the real-fixture-only names (`freeze\_real\_fixture`, `RefusalError`, `\_scan\_forbidden` and the real-fixture-only helpers) stay reachable via `backend.eval.golden\_fixtures` (re-export) AND are the SAME objects as the ones defined in the new `backend.eval.golden\_real\_fixtures` module ? so every existing import path (`gf.freeze\_real\_fixture` / `gf.RefusalError` / `gf.\_scan\_forbidden`) keeps working unchanged after a pure move.

\$6 honest-limits caveat: green here proves only that the public/used names resolve to the same objects and behave identically across both module paths ? behaviour-locking, NOT correctness.

```
import backend.eval.golden_fixtures as gf
import backend.eval.golden_real_fixtures as grf
```

Re-export identity ? the moved names are the SAME object via either module path.

```
def test_freeze_real_fixture_reexport_is_same_object():
    """``golden_fixtures.freeze_real_fixture`` is the exact object from ``golden_real_fixtures`` ?
    the re-export keeps the historical ``gf.freeze_real_fixture`` import path valid (same callable)."""
    assert gf.freeze_real_fixture is grf.freeze_real_fixture
```

```
def test_refusal_error_reexport_is_same_object():
    """``golden_fixtures.RefusalError`` is the exact class from ``golden_real_fixtures`` ?
    existing
    ``except gf.RefusalError`` / ``isinstance(e, gf.RefusalError)`` paths stay valid post-split."""
    assert gf.RefusalError is grf.RefusalError
```

```
def test_scan_forbidden_reexport_is_same_object():
    """``golden_fixtures._scan_forbidden`` is the exact function from
    ``golden_real_fixtures``."""
    assert gf._scan_forbidden is grf._scan_forbidden
```

```
def test_real_fixture_only_helpers_reexport_is_same_object():
    """The remaining real-fixture-only helpers moved alongside the public names resolve identically
```

```

via either module path (so any caller that reached them by either path is unaffected)."""
for name in ("_new_surrogate_slug", "_reset_trajectory_rows", "_reset_labels",
            "_SCHEMA_FILENAME_LITERALS"):
    assert getattr(gf, name) is getattr(grf, name), f"{name} is not the same object across
modules"

```

RefusalError still a ValueError subclass via BOTH module paths (behaviour-locking).

```

def test_refusal_error_is_value_error_subclass():
    """`RefusalError` remains a `ValueError` subclass (existing ``except ValueError`` paths
catch
it) ? pinned via both module paths since they are the same class."""
    assert issubclass(gf.RefusalError, ValueError)
    assert issubclass(grf.RefusalError, ValueError)

```

Shared primitives the real-fixture module depends on are still reached FROM golden_f

```

def test_real_module_reuses_shared_primitives_from_golden_fixtures():
    """The real-fixture module imports the shared primitives FROM ``golden_fixtures`` (not a
copy):
the constants/helpers it relies on resolve to the same objects, proving a pure move with a
single
source of truth (no duplicated logic/thresholds)."""
    for name in ("CURRENT_SCHEMA", "FIXTURES_DIR", "PINNED_VT", "PINNED_MERGE_GAP",
                "PINNED_MIN_WINDOW", "PINNED_MAX_WINDOW", "FORBIDDEN_KEYS", "_MD5_RE",
                "_LABEL_SLUG_RE", "_round", "write_trajectory_csv", "write_labels_csv",
                "_write_json", "replay_fixture", "_append_manifest_entry"):
        assert getattr(grf, name) is getattr(gf, name), (
            f"{name} should be the shared object from golden_fixtures, not a copy")

```

Behaviour smoke ? the re-exported gate behaves identically via either path.

```

def test_scan_forbidden_behaviour_identical_via_both_paths():
    """The re-exported ``scan_forbidden`` raises the same ``RefusalError`` on a forbidden key
regardless of which module path is used to call it (same object ? same behaviour)."""
    import pytest
    for fn in (gf._scan_forbidden, grf._scan_forbidden):
        with pytest.raises(gf.RefusalError):
            fn({'person': [1, 2, 3]}, where="x", source_paths=[])

```

21. user (2026-06-18T10:22:17.771Z)

File created successfully at: C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-5/tests/test_golden_fixtures_split.py (file state is current in your context ? no need to Read it back)

22. assistant (2026-06-18T10:22:22.319Z)

This test imports ``golden_real_fixtures``, which doesn't exist yet, so it will fail pre-refactor (collection error). That's expected for the cross-module identity contract. The pre-refactor characterization baseline is the EXISTING 54 tests passing (already confirmed). My new split-specific test pins the post-split contract and will be verified after the refactor.

Now let me perform the refactor. First, create the new ``golden_real_fixtures.py`` module with the moved symbols, importing shared primitives from ``golden_fixtures``.

23. assistant (2026-06-18T10:23:30.250Z)

```
"""Golden Regression Fixtures (P1 + M2) ? de-attribution / minimization + the refusal gate.
```

```

This module holds the real-fixture subsystem split out of `golden_fixtures` (which keeps
the
shared synthetic/replay primitives). It turns an OWNED, minors-free real trajectory + golden
labels
into a committable, de-attributed Tier-0 fixture, reusing M1's `replay_fixture` for the
snapshot.
(docs/GOLDEN_REGRESSION_FIXTURES.md §4c anti-attribution, §5 threat model, §7 row P1, §6 honest
limits.)
```

```

The shared primitives (constants, float/JSON helpers, `replay_fixture`, the CSV writers, the
manifest appender) live in `golden_fixtures` and are imported FROM it here ? there is a single
source of truth, no duplicated logic/thresholds. `golden_fixtures` re-exports the public names
defined here (`freeze_real_fixture`, `RefusalError`, `_scan_forbidden`) at the bottom of
that
module so every historical `golden_fixtures.<name>` import path stays valid.
```

```

§6 honest-limits caveat: a green suite proves only that the deterministic post-trajectory
transforms
are unchanged for the frozen cases ? characterization is behaviour-locking, NOT correctness.
"""
```

```
from __future__ import annotations
```

```
import os
import shutil
import tempfile
from pathlib import Path
from typing import Any, Dict, List, Optional, Sequence, Tuple
```

--- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only) -----

```
from backend.eval.gt_loader import load_gt_intervals
from backend.eval.regression import gt_hash
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
```

--- shared primitives, imported FROM golden_fixtures (single source of truth) -----

```
from backend.eval.golden_fixtures import (
    CURRENT_SCHEMA,
    FIXTURES_DIR,
    FORBIDDEN_KEYS,
    PINNED_MAX_WINDOW,
    PINNED_MERGE_GAP,
    PINNED_MIN_WINDOW,
    PINNED_VT,
    Interval,
    _LABEL_SLUG_RE,
    _MD5_RE,
    _append_manifest_entry,
    _round,
    _write_json,
    replay_fixture,
```

```

    write_labels_csv,
    write_trajectory_csv,
)

```

=====

P1 ? De-attribution + minimization: OWNED real trajectory+labels ? committable Tier-0 fixture, reusing the SAME replay_fixture snapshot. (docs/GOLDEN_REGRESSION_FIXTURES §4c anti-attribution, §5 threat model, §7 row P1, §6 honest limits.)

=====

```

class RefusalError(ValueError):
    """The provenance/biometric refusal gate (M2) rejected an unsafe freeze.

    A subclass of ``ValueError`` (so existing ``except ValueError`` paths still catch it) that
    names *why* the unsafe public-fixture path was refused. The refusal is the code-enforced
    provenance gate (§4c "provenance hard-fail gate", §7 ?): the owner-recorded / minors-
    excluded
    / license flags are human-asserted, but the gate makes the unsafe path impossible to reach
    *silently* ? a fixture cannot be written unless all three are affirmatively set.
    """

def _new_surrogate_slug() -> str:
    """An opaque RANDOM surrogate slug ? ``fx_<16 hex>`` from ``os.urandom`` (§4c).

    Deliberately NOT the MD5 ``video_id`` and NOT ``HMAC(salt, video_id?name)`` ? a random,
    NON-reversible id with no algebraic link to the source, so it can't be brute-forced over a
    tiny known roster even if a salt leaked. The surrogate?source map (if kept) lives off-git.
    """
    return f"fx_{os.urandom(8).hex()}"

#: The fixed schema filenames the writers ALWAYS emit (as ``paths`` literals). A source-path
token
#: that EXACTLY equals one of these is NOT a leak (the bytes would be written regardless), so it
is
#: excluded. Matched by exact membership, NEVER substring containment ? a substring rule would
let
#: a real source named ``label.csv`` (stem ``label`` ? ``labels.csv``) leak unnoticed (red-team
#4).
_SCHEMA_FILENAME_LITERALS: Tuple[str, ...] = (
    "trajectory", "trajectory.csv", "labels", "labels.csv", "expected", "expected.json",
    "provenance", "provenance.json",
)

def _scan_forbidden(text: str, *, where: str, source_paths: Sequence[str]) -> None:
    """POSITIVE privacy assertion (§4c, red-team ?): raise if ``text`` leaks anything
    attributable.

    Scans the written JSON *text* for (a) any ``FORBIDDEN_KEYS`` biometric/identity substring,
    (b) a 32-hex MD5 (the ``video_id`` shape), or (c) any identifying token of a source path
    (full path / basename / stem) ? excluding the fixed schema filename literals the writers
    always emit. The trajectory is shuttle-only by nature; this guards against a *regression*
    re-introducing a person/audio/pose stream or an id/path back-channel ? never the silent
    default.
    """
    low = text.lower()
    for key in FORBIDDEN_KEYS:

```

```

    if key in low:
        raise RefusalError(
            f"{where}: forbidden key substring {key!r} found in written output ? "
            f"Tier-0 fixtures are shuttle-only; person/pose/gait/face/audio/player MUST NOT
"
            f"appear (docs/GOLDEN_REGRESSION_FIXTURES.md §4c).")
m = _MD5_RE.search(text)
if m:
    raise RefusalError(
        f"{where}: a 32-hex MD5 ({m.group(0)!r}) found in written output ? that is the "
        f"video_id shape and an attribution back-channel; never write it (§4c/§5).")
for sp in source_paths:
    if not sp:
        continue
    p = Path(sp)
    for token in (sp, p.name, p.stem):
        t = (token or "").strip().lower()
        # Skip ONLY a token that EXACTLY equals a schema-literal the writers always emit
        # (e.g. a source named ``trajectory.csv``) ? that collision is not a leak. We must
NOT
        # skip on substring containment: a token like ``label`` or ``traj`` is a SUBSTRING
of
        # ``labels.csv``/``trajectory.csv``, so substring-skip would silently miss a real
source
        # named ``label.csv`` leaking its stem (red-team #4). Exact membership only.
        if not t or t in _SCHEMA_FILENAME_LITERALS:
            continue
        if t in low:
            raise RefusalError(
                f"{where}: source-path token {token!r} found in written output ? the source
"
                f"filename/path must never be persisted (§4c metadata strip).")

def _reset_trajectory_rows(points: Sequence[Any], t0_frame: int
    ) -> List[Tuple[int, int, float, float]]:
    """Shift every ``TrajectoryPoint`` frame by ``-t0_frame`` ? ``(Frame,Visibility,X,Y)`` rows.
    Visibility/X/Y are passed through untouched (the shuttle geometry is metric-invariant under
a
    pure time shift); only the absolute frame index moves toward 0, severing the match timeline.
    """
    rows: List[Tuple[int, int, float, float]] = []
    for p in points:
        rows.append((int(p.frame) - t0_frame, 1 if p.visible else 0, float(p.x), float(p.y)))
    rows.sort(key=lambda r: r[0])
    return rows

def _reset_labels(gts: Sequence[Interval], t0_sec: float) -> List[Interval]:
    """Shift every label by ``-t0_sec`` (same offset as the trajectory ? metric-invariant).
    ``t0_sec`` MUST already be quantized to ``_FLOAT_ROUND_DP`` decimals by the caller. Each
label
    endpoint is ALSO rounded to 6dp BEFORE subtracting, so the shift is the difference of two
exact
    6dp quantities ? no double-rounding (``round6(orig - round6(t0))`` vs the no-reset
    ``round6(orig)`` accumulated error; red-team #2). This keeps the RAW reset-vs-no-reset score
    delta at the float-error floor (~3.3e-7, well within ``_FLOAT_TOL``); see
    ``freeze_real_fixture``
    for why the SERIALIZED 6dp boundary error can still differ by one 6dp ULP on a sub-second
    offset (the grid, not a real metric divergence)."""
    return [(_round(s) - t0_sec, _round(e) - t0_sec) for s, e in gts]

```

```

def freeze_real_fixture(
    label: str,
    trajectory_csv_path: Any,
    labels_csv_path: Any,
    *,
    license: str,
    minors_free: bool,
    owned: bool,
    fps: float,
    frame_width: float,
    fixtures_dir: Path = FIXTURES_DIR,
    slug: Optional[str] = None,
    time_origin_reset: bool = True,
    source_note: str = "",
) -> Dict[str, Any]:
    """Turn an OWNED, minors-free real trajectory + golden labels into a committable, de-
attributed
    Tier-0 fixture ? reusing M1's ``replay_fixture`` for the snapshot. ($4c / $5 / $7 row P1.)

    The refusal gate (M2) makes the unsafe path impossible to reach silently: a fixture is
emitted
    ONLY when ``minors_free is True`` AND ``owned is True`` AND ``license`` is a non-blank
string,
    AND ``label`` is a strict slug (lowercase alnum + ``_``/``-`` ? a free-text identifier like
a
    player/event name is refused, red-team #1). The committed bytes carry NO filename / video_id
/
    match / player / capture-date / operator free-text ? only an opaque random surrogate slug,
the
    shuttle-only trajectory (time-origin reset by default), the reset labels, and a
    ``kind="cleared_real"`` provenance block. A positive privacy assertion re-reads the written
    ``expected.json`` + ``provenance.json`` and refuses if anything attributable slipped in.

    ``source_note`` is **accepted but NOT persisted** ? operator free-text is an attribution
back-channel ``_scan_forbidden`` cannot fully police (an identifying note that happens to
contain no FORBIDDEN_KEY / MD5 / source-path token would slip through), so it never reaches
the
    committed ``provenance.json`` (red-team #1). It is still scanned defensively and the flag is
retained for CLI compatibility / future logging.

    NOTE ($6 honest limits): the random surrogate + reset timeline make the fixture non-
attributable
    only *relative to the unpublished source* ? it is NOT absolutely anonymous (EDPS v. SRB).
De-
    attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off (P2) and
the
    owner-asserted Fork-A / minors / biometric-exclusion flags.

    The fixture is written into a temp dir and promoted (``os.replace``) to the final slug dir
ONLY
    after the privacy scan passes ? a refused / failed freeze leaves NO partial fixture dir
    (red-team #3).

    Returns the recomputed ``expected.json`` snapshot dict (identical to ``replay_fixture``).
    """
    # --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail ($4c / $7 ?) -----
    reasons: List[str] = []
    if minors_free is not True:
        reasons.append("minors_free must be True (DPDP child = under 18; behavioural monitoring
"
                        "of a minor is prohibited ? §3, guardrail 'exclude minors')")
    if owned is not True:
        reasons.append("owned must be True (only owner-recorded Fork-A source is safe to commit;
"
                        "broadcast/scraped = Fork B, do NOT commit ? §3 D1)")

```

```

if not (isinstance(license, str) and license.strip()):
    reasons.append("license must be a non-blank string (per-record provenance tag ? $4c "
        "'provenance hard-fail gate')")
# label is committed verbatim into provenance ? it MUST be a strict slug, never operator
free
# text (a name/event), which _scan_forbidden cannot reliably catch (red-team #1 free-text
leak).
if not (isinstance(label, str) and _LABEL_SLUG_RE.match(label)):
    reasons.append(
        f"label must match the strict slug charset {_LABEL_SLUG_RE.pattern!r} (lowercase "
        f"alnum + '_'/'-', no spaces/punctuation) ? a free-text identifier (player/event "
        f"name) is an attribution leak; got {label!r} ($4c metadata strip, red-team #1)")
if reasons:
    raise RefusalError(
        "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution does
NOT "
        "cure provenance/licensing; this gate sits behind counsel sign-off, $6/P2):\n - "
        + "\n - ".join(reasons))

traj_path = Path(str(trajjectory_csv_path))
labels_path = Path(str(labels_csv_path))

# --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source name ---
out_slug = slug if slug is not None else _new_surrogate_slug()

# --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient) ----
points = TrackNetRunner.parse_trajectory_csv(str(traj_path))
if not points:
    raise ValueError(f"{traj_path}: no parseable trajectory rows (Frame,Visibility,X,Y)")
gts = load_gt_intervals(str(labels_path), strict=True)

# --- (d) consistent time-origin reset (default ON; metric-invariant within _FLOAT_TOL, $4c)
--
# Subtract the SAME offset from BOTH the trajectory frames and the label seconds:
metrics.score
# is built from pure differences (interval_iou, |start-start|, |end-end|), so a pure time
shift
# leaves the score essentially unchanged ? only the absolute timeline is severed. To keep
the
# two shifts a consistent quantity (and avoid double-rounding past _FLOAT_TOL ? red-team
#2),
# t0_sec is ROUNDED to 6dp BEFORE subtracting (and _reset_labels rounds each endpoint
first), so
# the RAW reset-vs-no-reset score delta sits at the float-error floor (~3.3e-7 ?
_FLOAT_TOL).
#
# HONEST LIMIT (not a bug): the trajectory shifts by an INTEGER frame count while labels
shift
# in SECONDS, and 6dp-rounding is NOT additive (round6(8/fps) - round6(7/fps) ?
round6(1/fps)),
# so the *serialized* 6dp boundary error (mean_start/end_error) can differ by ONE 6dp ULP
# (1e-6) on a sub-second offset. That is the serialization grid, not a metric divergence ?
the
# score is bit-identical for whole-second offsets and within _FLOAT_TOL otherwise (test #2).
if time_origin_reset:
    t0_frame = min(int(p.frame) for p in points)
    t0_sec = _round(t0_frame / float(fps))
else:
    t0_frame, t0_sec = 0, 0.0
rows = _reset_trajectory_rows(points, t0_frame)
reset_gts = _reset_labels(gts, t0_sec)

# --- (d2) negative-label guard BEFORE any write (red-team #3a) -----
# A reset label that starts < 0 means the label timeline predates the earliest trajectory
# frame ? an inconsistent input that would crash the strict gt_loader on replay (negative

```

```

# start). Refuse loudly up-front so the partial-write path is never even reached.
neg = [(s, e) for (s, e) in reset_gts if s < 0]
if neg:
    raise RefusalError(
        f"REFUSED: label timeline predates the trajectory ? {len(neg)} reset label(s) have a
"
        f"negative start after the t0={t0_sec:.6f}s time-origin reset (e.g. {neg[0]!r}). The
"
        f"earliest label must not precede the earliest trajectory frame ($4c time-origin
reset).")

iou = 0.5
# NOTE (red-team #1b): operator free-text `source_note` is DELIBERATELY NOT a key in `prov`
?
# it never reaches the committed bytes (an identifying note is an attribution back-channel
# _scan_forbidden cannot fully police). The fixed boilerplate `note` constant below stays.
prov = {
    "slug": out_slug,
    "schema_version": CURRENT_SCHEMA,
    "kind": "cleared_real",
    "label": str(label),
    "license": license,
    "minors_free": True,
    "owned": True,
    "fps": _round(fps),
    "frame_width": _round(frame_width),
    "iou": _round(iou),
    "time_origin_reset": bool(time_origin_reset),
    "windowing": {
        "vt": PINNED_VT,
        "mg": PINNED_MERGE_GAP,
        "mwd": PINNED_MIN_WINDOW,
        "max_win": PINNED_MAX_WINDOW,
    },
    "paths": {
        "trajectory": "trajectory.csv",
        "labels": "labels.csv",
        "expected": "expected.json",
    },
    "gt_hash": gt_hash(reset_gts),
    "note": (
        "De-attributed REAL Fork-A fixture (owner-recorded, owned, minors-excluded,
biometric-"
        "free, shuttle-only). Opaque RANDOM surrogate slug; absolute timeline reset. NON-"
        "attributable only relative to the UNPUBLISHED source (EDPS v. SRB) ? never
'anonymous'. "
        "Tier-0 / CONTROL-only. $6: behaviour-locking, not correctness; sits behind counsel
(P2).",
    ),
}

# --- (e) write into a TEMP dir, replay, scan, and only THEN promote (red-team #3b) ----
# Everything is written under a sibling temp dir; the final slug dir is created by an atomic
# os.replace ONLY after the privacy scan passes. A refused/failed freeze therefore leaves NO
# partial fixture dir behind. source_note (if any) is scanned defensively but never written.
fdir = fixtures_dir / out_slug
fixtures_dir.mkdir(parents=True, exist_ok=True)
if fdir.exists():
    raise ValueError(f"{fdir}: fixture dir already exists ? refuse to clobber (choose a new
slug)")
tmp_parent = Path(tempfile.mkdtemp(prefix=f".{out_slug}.tmp.", dir=str(fixtures_dir)))
tmp_dir = tmp_parent / out_slug
tmp_dir.mkdir(parents=True, exist_ok=True)
try:
    write_trajectory_csv(rows, tmp_dir / "trajectory.csv")

```

```

write_labels_csv(reset_gts, tmp_dir / "labels.csv")
_write_json(tmp_dir / "provenance.json", prov)

expected = replay_fixture(tmp_dir) # REUSE M1 ? correct-by-construction snapshot
_write_json(tmp_dir / "expected.json", expected)

# --- (f) POSITIVE privacy assertion BEFORE promotion (scan written text + source_note)
---
source_paths = [str(traj_path), str(labels_path)]
_scan_forbidden((tmp_dir / "expected.json").read_text(encoding="utf-8"),
                 where=f"{out_slug}/expected.json", source_paths=source_paths)
_scan_forbidden((tmp_dir / "provenance.json").read_text(encoding="utf-8"),
                 where=f"{out_slug}/provenance.json", source_paths=source_paths)
# source_note is never persisted, but a leak in it is still a contract violation ? scan
it.
if source_note:
    _scan_forbidden(str(source_note),
                    where=f"{out_slug}/source_note(not-persisted)",
                    source_paths=source_paths)

# --- (g) PROMOTE atomically: temp slug dir ? final slug dir (scan passed) -----
os.replace(str(tmp_dir), str(fdir))
finally:
    # Clean up the temp parent whether we promoted or refused (a refusal leaves NO partial
    dir).
    shutil.rmtree(str(tmp_parent), ignore_errors=True)

# --- (h) append to the manifest with kind="cleared_real" (don't clobber synthetic) -----
_append_manifest_entry(prov, fixtures_dir)
return expected

```

24. user (2026-06-18T10:23:30.262Z)

File created successfully at: C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-5/backend/eval/golden_real_fixtures.py (file state is current in your context ? no need to Read it back)

25. assistant (2026-06-18T10:23:40.476Z)

I notice the original `golden\_fixtures.py` doesn't import `Interval` ? it defines it as `Interval = Tuple[float, float]` at module level. I imported `Interval` from `golden\_fixtures` which is fine since it's a module-level name. Let me verify `Interval` is importable. It's defined as `Interval = Tuple[float, float]` at line 72 ? yes, importable.

Now remove the moved blocks from `golden\_fixtures.py` and add the bottom re-export. First, remove the `RefusalError` class through the end of `freeze\_real\_fixture` (lines 500-784), but keep `\_manifest\_entry\_for` and `\_append\_manifest\_entry` which are shared (used by `freeze\_real\_fixture` AND are manifest helpers).

Wait ? `\_append\_manifest\_entry` and `\_manifest\_entry\_for` are after `freeze\_real\_fixture` in the original. They're used by `freeze\_real\_fixture` but are general manifest helpers (shared). They should stay in `golden\_fixtures`. Let me remove only the P1 real-fixture block (lines 500-784: `RefusalError` through end of `freeze\_real\_fixture`), leaving the section header comment and the manifest helpers.

Let me look at the exact boundaries. The P1 section header is at lines 493-497. `RefusalError` starts at 500. `freeze\_real\_fixture` ends at 784. Then `\_manifest\_entry\_for` starts at 787. I'll remove lines 500-785 (the class + helpers + freeze_real_fixture + the trailing blank line) but keep the P1 section header comment since it documents what `freeze\_real\_fixture` does ? actually no, the header now describes code that moved. Let me replace the section header + moved code with nothing, but `\_manifest\_entry\_for`/_`\_append\_manifest\_entry` need their own context.

Let me do the edit precisely. Remove from the P1 section header through the end of `freeze\_real\_fixture`.

26. user (2026-06-18T10:24:42.639Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-5/backend/eval/golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

27. assistant (2026-06-18T10:24:45.893Z)

Now I need to add the bottom re-export and remove now-unused imports from `golden\_fixtures.py`. Let me check which imports became unused. The moved code used: `os`, `shutil`, `tempfile`, `TrackNetRunner`. Let me check if `golden\_fixtures.py` still uses these after removal.

28. user (2026-06-18T10:24:47.525Z)

33: takes only the 3-tuple `(vt, mg, mwd)` and
`TrackNetRunner.trajectory\_to\_action\_windows`

29. assistant (2026-06-18T10:24:53.169Z)

All four (`os`, `shutil`, `tempfile`, `TrackNetRunner`) are now unused in `golden\_fixtures.py` (the only `TrackNetRunner` hit is inside a docstring). Let me verify the other potentially-affected imports remain used: `gt\_hash`, `load\_gt\_intervals`, and typing names.

30. user (2026-06-18T10:24:54.751Z)

gt_hash: 5
load_gt_intervals: 3
Sequence: 5
Optional: 5

31. assistant (2026-06-18T10:24:59.216Z)

`gt\_hash`, `load\_gt\_intervals`, `Sequence`, `Optional` are all still used (they appear in `replay\_fixture`, the CSV writers, etc.). Now let me check the import block of golden_fixtures and remove `os`, `shutil`, `tempfile`, and the `TrackNetRunner` import.

32. user (2026-06-18T10:25:00.066Z)

```
50     from __future__ import annotations
51
52     import argparse
53     import json
54     import math
55     import os
56     import re
57     import shutil
58     import sys
59     import tempfile
60     from dataclasses import dataclass
61     from pathlib import Path
62     from typing import Any, Dict, List, Optional, Sequence, Tuple
63
64     # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
65     -----
66     from backend.eval import distill_local
67     from backend.eval.experiment import CONTROL, VideoCandidates, predict
68     from backend.eval.gt_loader import load_gt_intervals
69     from backend.eval.metrics import score
70     from backend.eval.regression import gt_hash
71     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
```

33. assistant (2026-06-18T10:25:03.118Z)

Let me verify `os`, `re`, `shutil`, `tempfile` usage in the remaining file before removing. `re` is used by `\_MD5\_RE`/`\_LABEL\_SLUG\_RE` (shared, stay). Let me confirm `os`, `shutil`, `tempfile` are truly unused now.

34. user (2026-06-18T10:25:04.952Z)

```
os\ .-> 1
shutil\ .-> 0
tempfile\ .-> 0
re\ .-> 8
```

35. assistant (2026-06-18T10:25:07.617Z)

`os.` has 1 hit ? let me check whether it's a real usage or a docstring/comment.

36. user (2026-06-18T10:25:08.790Z)

246: """The committed synthetic scenarios. Deterministic; chosen so each exercises a distinct

37. assistant (2026-06-18T10:25:13.938Z)

That `os.` hit is inside the phrase "each exercises a distinct" (the "os." in "exercises" ? actually it's "exerci**s**e**s** ... " ? the regex `os\.` matched something). Let me check ? line 246 doesn't contain `os.` followed by code. The match is likely "scenarios." ? "scenari**os.**" Yes ? "scenarios." contains "os." That's a docstring word, not a usage. So `os`, `shutil`, `tempfile` are all genuinely unused now. Remove them.

`TrackNetRunner` import ? only used in docstring now. Remove it too. Let me remove the four unused imports.

38. user (2026-06-18T10:25:18.170Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-5/backend/eval/golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

39. assistant (2026-06-18T10:25:25.033Z)

Now add the bottom re-export. It must go at the very end of the file, after all shared primitives are defined, but BEFORE the `if \_\_name\_\_ == "\_\_main\_\_":` guard ? actually it can go right before the CLI section or at the very bottom. The cleanest place is just before the `if \_\_name\_\_` guard, after `main()` is defined. But placement-wise, the re-export at the bottom ensures all shared primitives (`replay\_fixture`, `\_append\_manifest\_entry`, `\_round`, etc.) are already defined when `golden\_real\_fixtures` is imported. Let me read the bottom of the file.

40. user (2026-06-18T10:25:26.797Z)

```
820         raise SystemExit(main())
821
```

41. user (2026-06-18T10:25:30.973Z)

```
808         return 2
809         print(f"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}'")
810         f"(kind=cleared_real) -> {fixtures_dir / expected['slug']}")
811         print(" $6: behaviour-locking, NOT correctness; non-attributable only relative
to the "
812             "UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
(P2).")
813         return 0
814
815         # --check
```

```

816         return _cmd_check(fixtures_dir)
817
818
819     if __name__ == "__main__":
820         raise SystemExit(main())
821

```

42. assistant (2026-06-18T10:25:42.410Z)

Now I need to confirm where `freeze_real_fixture`, `RefusalError`, `_scan_forbidden` are referenced in `golden_fixtures.py`'s `main()` CLI. The CLI calls `freeze_real_fixture` and `except RefusalError`. So the re-export MUST be available at module-execution time before `main()` runs. Since `main()` only runs when called (not at import), placing the re-export at the bottom (before the `if __name__` guard) works for the CLI path too ? by the time `main()` is invoked, the module is fully loaded.

However, there's a circular import concern: `golden_fixtures` imports `golden_real_fixtures` at the bottom, and `golden_real_fixtures` imports `golden_fixtures` at the top. When `golden_fixtures` is imported first: it runs top-to-bottom, defines all shared primitives, then hits the bottom re-export `from golden_real_fixtures import ...`. That triggers `golden_real_fixtures` to import, which does `from golden_fixtures import ...` ? at this point `golden_fixtures` is partially initialized but all the needed names (which are above the bottom re-export) ARE defined. So it resolves.

When `golden_real_fixtures` is imported first: it does `from golden_fixtures import ...` at the top, which triggers full `golden_fixtures` import including the bottom re-export `from golden_real_fixtures import ...` ? but `golden_real_fixtures` is partially initialized (only its top imports ran, the classes/functions below aren't defined yet). This would FAIL because `freeze_real_fixture` etc. aren't defined in `golden_real_fixtures` yet when `golden_fixtures`'s bottom re-export runs.

I need to handle BOTH import orders. The robust fix is to make the bottom re-export lazy OR restructure. Let me think about the cleanest approach that's still a pure move.

The issue: bottom-of-`golden_fixtures` eager re-export breaks the `golden_real_fixtures`-first order. Let me verify this is a real problem by testing both orders after I add the eager version, then decide. Actually, let me reason it through and pick the lazy `__getattr__` re-export which handles both orders cleanly.

Let me add a module-level `__getattr__` (PEP 562) at the bottom of `golden_fixtures.py` that lazily imports from `golden_real_fixtures`. This avoids the cycle entirely because the import only happens on first attribute access, by which time both modules are fully loaded.

43. user (2026-06-18T10:25:50.128Z)

The file `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-5/backend/eval/golden_fixtures.py` has been updated successfully. (file state is current in your context ? no need to Read it back)

44. assistant (2026-06-18T10:25:54.778Z)

Now, the CLI `main()` in `golden_fixtures.py` references `freeze_real_fixture` and `RefusalError` directly as bare names. With `__getattr__`, bare-name references inside the module do NOT trigger `__getattr__` (it only fires for module-attribute access from outside, or `getattr(module, ...)`). Inside `main()`, `freeze_real_fixture(...)` is a name lookup in the module globals ? which will now `NameError` because they're no longer defined as module globals.

Let me check the CLI references and fix them to use the lazy import explicitly.

45. user (2026-06-18T10:25:56.181Z)

115:#: attribution back-channel that `_scan_forbidden` does NOT catch (it isn't a FORBIDDEN_KEY / MD5

798: expected = freeze_real_fixture(

```
805:         except RefusalError as e:
831:     "RefusalError", "freeze_real_fixture", "_scan_forbidden", "_new_surrogate_slug",
```

46. assistant (2026-06-18T10:25:59.552Z)

Lines 798 and 805 in `main()` use bare names `freeze\_real\_fixture` and `RefusalError`. These would `NameError`. I need the CLI to reference the real-fixture module. The cleanest pure-move approach: in the `args.freeze\_real` branch, import `golden\_real\_fixtures` and call through it. Let me read that block.

47. user (2026-06-18T10:26:00.492Z)

```
789
790     if args.freeze_real:
791         missing = [n for n, v in (("--trajectory", args.trajectory),
792                                 ("--labels", args.labels),
793                                 ("--fps", args.fps),
794                                 ("--frame-width", args.frame_width)) if v is None]
795     if missing:
796         ap.error("--freeze-real requires " + ", ".join(missing))
797     try:
798         expected = freeze_real_fixture(
799             args.label, args.trajectory, args.labels,
800             license=args.license_ if args.license_ is not None else "",
801             minors_free=args.minors_free, owned=args.owned,
802             fps=args.fps, frame_width=args.frame_width,
803             fixtures_dir=fixtures_dir, slug=args.slug,
804             time_origin_reset=args.time_origin_reset, source_note=args.source_note)
805     except RefusalError as e:
806         # The code-enforced provenance gate ($4c / $7 ?): refuse loudly, exit non-
807         zero.
808         print(f"[golden-fixtures] REFUSED (--freeze-real provenance gate):\n{e}",
809             file=sys.stderr)
810         return 2
811     print(f"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}'
812     "
813         f"(kind=cleared_real) -> {fixtures_dir / expected['slug']}")
814     print(" $6: behaviour-locking, NOT correctness; non-attributable only relative
815     to the "
816         "UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
817     (P2).")
818     return 0
```

48. assistant (2026-06-18T10:26:04.108Z)

I'll update the CLI branch to import `golden\_real\_fixtures` lazily and call through it. This is a behavior-preserving change ? the same function and exception class (they're the same objects), just referenced through the module that now defines them. The output strings, exit codes, and logic are unchanged.

49. user (2026-06-18T10:26:10.799Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-5/backend/eval/golden_fixtures.py has been updated successfully. (file state is current in your context ? no need to Read it back)

50. assistant (2026-06-18T10:26:14.071Z)

Now let me verify both import orders resolve cleanly, and that the re-export identity holds.

51. user (2026-06-18T10:26:22.781Z)

```
=== order A: golden_fixtures first ===
OK order A
```

```
=== order B: golden_real_fixtures first ===
OK order B
```

52. assistant (2026-06-18T10:26:25.472Z)

Both import orders resolve cleanly and re-export identity holds. Now run the existing golden tests plus my new split test.

53. user (2026-06-18T10:26:30.763Z)

```
..... [100%]
61 passed in 2.74s
```

54. assistant (2026-06-18T10:26:34.329Z)

61 passed (54 existing + 7 new split tests). Now verify the CLI `--freeze-real` path still works end-to-end and the `--check` path works (since `\_cmd\_check` uses `replay\_fixture` which stays in `golden\_fixtures`).

55. user (2026-06-18T10:26:42.151Z)

```
=== --check ===
[ok] clean_single_rally: 1 candidates, 1 segments, F1=1.000000
[ok] multi_rally_with_deadtime: 2 candidates, 2 segments, F1=1.000000
[ok] occlusion_break: 2 candidates, 2 segments, F1=1.000000
[golden-fixtures] 3/3 fixtures match (synthetic-tier = plumbing correctness, not field quality ?
$6)
=== --freeze-real refusal (no flags) ===
[golden-fixtures] REFUSED (--freeze-real provenance gate):
REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution does NOT cure
provenance/licensing; this gate sits behind counsel sign-off, $6/P2):
  - minors_free must be True (DPDP child = under 18; behavioural monitoring of a minor is
prohibited ? $3, guardrail 'exclude minors')
  - owned must be True (only owner-recorded Fork-A source is safe to commit; broadcast/scraped =
Fork B, do NOT commit ? $3 D1)
exit=0
```

56. assistant (2026-06-18T10:26:45.402Z)

The CLI refusal path works through the new module (the `RefusalError` is caught, prints REFUSED, returns 2 ? the `exit=0` is the `head` pipe's exit, not the python exit). Both CLI paths behave identically.

Now let me run the FULL gate. First the full suite.

57. user (2026-06-18T10:28:01.227Z)

```
=====
Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
obsreport (optional reporting dep): available
=====
.....SS..... [ 8%]
..... [ 16%]
..... [ 24%]
..... [ 33%]
..... [ 41%]
..... [ 49%]
..... [ 57%]
..... [ 66%]
..... [ 74%]
.....SSSSS..... [ 82%]
..... [ 90%]
..... [ 99%]
```

```
..... [100%]
864 passed, 7 skipped in 71.35s (0:01:11)
=====
RESULT: PASS - all tests green
=====
```

58. assistant (2026-06-18T10:28:04.603Z)

Full suite green: 864 passed, 7 skipped. Now ruff and mypy.

59. user (2026-06-18T10:28:41.796Z)

```
=== ruff ===
All checks passed!
=== mypy ===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 108 source files
```

60. assistant (2026-06-18T10:28:44.418Z)

Ruff clean, mypy success (only informational notes, no errors). Now the coverage gate.

61. user (2026-06-18T10:30:17.228Z)

backend\providers__init__.py	16	1	94%
backend\providers\base.py	9	1	89%
backend\providers\gemini.py	98	19	81%
backend\reporting__init__.py	26	2	92%
backend\reporting\harvest.py	111	8	93%
backend\reporting\run_signals.py	14	0	100%
backend\results.py	27	0	100%
backend\sports__init__.py	17	1	94%
backend\sports\badminton.py	13	1	92%
backend\sports\base.py	12	3	75%
backend\tools__init__.py	0	0	100%
backend\tools\export.py	75	2	97%
backend\tools\rally_slicer.py	104	47	55%
backend\utils__init__.py	0	0	100%
backend\utils\geometry.py	29	0	100%
backend\utils\hashing.py	9	0	100%
backend\utils\logging_setup.py	11	0	100%
backend\utils\paths.py	27	3	89%
backend\utils\run_telemetry.py	126	12	90%
backend\utils\video.py	25	3	88%
training__init__.py	0	0	100%
training\gen0__init__.py	0	0	100%
training\gen0\harness.py	108	11	90%

TOTAL	8105	1543	81%

Required test coverage of 70% reached. Total coverage: 80.96%

864 passed, 7 skipped, 7 warnings in 88.39s (0:01:28)

```
=====
RESULT: PASS - all tests green
=====
```

62. assistant (2026-06-18T10:30:20.506Z)

All four gates green. Coverage is 80.96% ? let me check the per-file coverage of the two split files specifically to confirm no regression on the moved code.

63. user (2026-06-18T10:31:53.579Z)

backend\eval\golden_fixtures.py	353	47	87%
backend\eval\golden_real_fixtures.py	92	2	98%

64. assistant (2026-06-18T10:31:58.161Z)

The split files are well-covered: `golden\_fixtures.py` 87%, `golden\_real\_fixtures.py` 98%. Coverage 80.96% is at the ~80.9% base ? no regression.

Now let me compare against the base to confirm coverage didn't drop. The base was ~80.9%; we're at 80.96%. Let me capture the diffstat and verify the working tree is exactly the files I intended.

65. user (2026-06-18T10:31:59.790Z)

```
M backend/eval/golden_fixtures.py
?? backend/eval/golden_real_fixtures.py
?? tests/test_golden_fixtures_split.py
=== diffstat (tracked) ===
 backend/eval/golden_fixtures.py | 327 ++++-----
 1 file changed, 31 insertions(+), 296 deletions(-)
```

66. assistant (2026-06-18T10:32:03.836Z)

The `artifacts/` and `docs/KHELSUTRA\_PRODUCT\_OVERVIEW.md` from the initial git status are not present here (this is a clean worktree). Exactly three changes: modified `golden\_fixtures.py`, new `golden\_real\_fixtures.py`, new test file.

Let me re-verify HEAD/branch right before commit per the branching-safety protocol, and confirm only my commits will be on the branch.

67. user (2026-06-18T10:32:05.441Z)

```
refactor/golden-fixtures-real-split
e67add1 refactor(api): extract chunked-upload island from main into backend.api.uploads router
(#228)
=== commits ahead of origin/master ===
```

68. assistant (2026-06-18T10:32:10.434Z)

Branch is correct, HEAD is at origin/master (no sibling commits ahead). Let me do a final re-verification that the gates are all green together one more time was already done. Let me also double-check there are no other references in the codebase that imported the moved names directly (only via `golden\_fixtures` re-export or the new module). I already confirmed only the test files and docs reference them. Let me also verify the new module passes ruff/mypy individually (it's covered by the `` and `backend` runs above, but confirm no per-file issue).

Now stage explicit paths and commit.

69. user (2026-06-18T10:32:11.695Z)

```
warning: in the working copy of 'backend/eval/golden_real_fixtures.py', LF will be replaced by
CRLF the next time Git touches it
warning: in the working copy of 'tests/test_golden_fixtures_split.py', LF will be replaced by
CRLF the next time Git touches it
M backend/eval/golden_fixtures.py
A backend/eval/golden_real_fixtures.py
A tests/test_golden_fixtures_split.py
```

70. assistant (2026-06-18T10:32:15.127Z)

All three staged. The CRLF warnings are expected (repo runs `core.autocrlf=true`). Now commit.

71. user (2026-06-18T10:32:23.552Z)

```
[refactor/golden-fixtures-real-split 727a421] refactor(eval): split real-fixture subsystem into
golden_real_fixtures (pure move)
3 files changed, 461 insertions(+), 296 deletions(-)
create mode 100644 backend/eval/golden_real_fixtures.py
create mode 100644 tests/test_golden_fixtures_split.py
```

72. assistant (2026-06-18T10:32:26.603Z)

Committed. Now push and open the PR.

73. user (2026-06-18T10:32:29.888Z)

```
remote:
remote: Create a pull request for 'refactor/golden-fixtures-real-split' on GitHub by visiting:
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/refactor/golden-
fixtures-real-split
remote:
branch 'refactor/golden-fixtures-real-split' set up to track 'origin/refactor/golden-fixtures-
real-split'.
To https://github.com/avidullu/badminton-highlight-indexer.git
* [new branch]      refactor/golden-fixtures-real-split -> refactor/golden-fixtures-real-split
```

74. user (2026-06-18T10:32:49.179Z)

<https://github.com/avidullu/badminton-highlight-indexer/pull/233>

75. assistant (2026-06-18T10:32:53.664Z)

PR opened: <https://github.com/avidullu/badminton-highlight-indexer/pull/233>

Let me capture the precise gate counts for the structured output (suite pass/skip already captured: 864 passed, 7 skipped).

76. user (2026-06-18T10:32:55.457Z)

```
backend/eval/golden_fixtures.py      | 327 ++++-----
backend/eval/golden_real_fixtures.py | 343 ++++++
tests/test_golden_fixtures_split.py  | 87 +++++++
3 files changed, 461 insertions(+), 296 deletions(-)
```

77. assistant (2026-06-18T10:32:58.110Z)

All four gates passed, coverage held, PR opened. Let me return the structured result.

78. user (2026-06-18T10:33:16.569Z)

Structured output provided successfully

79. assistant (2026-06-18T10:33:24.794Z)

I've completed the golden-fixtures-split tech-debt item. Summary:

```
**Refactor:** Split the real-fixture subsystem (P1 + M2) out of the 1100-line
`backend/eval/golden_fixtures.py` into a new `backend/eval/golden_real_fixtures.py`, mirroring
the existing test split. Moved `freeze_real_fixture`, `RefusalError`, `_scan_forbidden`, and
their real-fixture-only helpers verbatim; the new module imports shared primitives from
`golden_fixtures` (single source of truth).
```

****Discipline followed:**** Branched from `origin/master`, confirmed the 54 existing characterization tests pass on unchanged code first, then did the behavior-preserving move, then re-verified the same tests pass plus 7 new re-export-identity tests.

****Circular-import handling:**** Used a lazy PEP-562 `\_\_getattr\_\_` re-export (the task suggested an eager bottom-of-file import, but that breaks the `golden\_real\_fixtures`-first import order ? the lazy hook resolves BOTH orders, both verified). Every historical `golden\_fixtures.<name>` path resolves to the same object as `golden\_real\_fixtures.<name>`.

****Gate (all green, coverage held at 80.96% vs ~80.9% base):****

- Suite: 864 passed, 7 skipped
- ruff: All checks passed
- mypy: Success, no issues in 108 files
- coverage: 80.96% (new module 98%, golden_fixtures 87%)

PR opened: <https://github.com/avidullu/badminton-highlight-indexer/pull/233> (not merged, as instructed).